



ARISTOTLE UNIVERSITY OF THESSALONIKI

Enhancing Personalized Travel Assistance with Retrieval-Augmented Generation: A Context-Aware Approach

by

Marios Christos Malamatinos - 133

A thesis submitted in partial fulfillment for the
Postgraduate degree

in the
Faculty of Sciences
School of Informatics

Supervising Professor: Dr. Athena Vakali

28/02/2025

Declaration of Authorship

I, Marios Christos Malamatinos, declare that this thesis titled, "Enhancing Personalized Travel Assistance with Retrieval-Augmented Generation: A Context-Aware Approach" and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date:

The important thing is not to stop questioning. Curiosity has its own reason for existing.

Albert Einstein

Abstract

Faculty of Sciences
School of Informatics

Postgraduate Degree

by Marios Christos Malamatinos - 133

This thesis introduces an advanced Retrieval-Augmented Generation (RAG) framework designed to enhance tourism related recommendation systems. The proposed system integrates multiple state-of-the-art retrieval methodologies, such as Hybrid Vector Retrieval, AutoMerge, HyDE, and GraphRAG each evaluated through extensive experimentation on a synthetic domain-specific dataset for improving the accuracy and relevance of tourism-related information. By leveraging domain-specific knowledge the framework aims to provide dynamic, personalized, and contextual recommendations for travelers. This work also incorporates a personalization component that dynamically constructs a user profile through prompt engineering, enhancing the system's ability to tailor responses based on individual characteristics and preferences such as age range, nationality, travel group, purpose of travel, etc. thus delivering a personalized travel assistance experience. Moreover, it emphasizes on an End-to-End automated, open-source approach, making it adaptable and accessible for future research and real-world applications in tourism. To evaluate the effectiveness of the framework, extensive experimentation was conducted using domain-specific generated dataset, with the help of retrieval and generative quality metrics. The results of this research clearly demonstrate that the HyDE architecture consistently outperforms all the others both for locally hosted, lightweight open source models (Llama3.2 3B Instruct, BGE-M3) as well as for models provided by external Large Language Model (LLM) providers such as OpenAI (GPT-4o-mini, text-embedding-3-small). Additionally, experiments comparing the local and the OpenAI models for the HyDE approach, reveal that locally hosted BGE-M3 outperforms text-embedding-3-small model in retrieval while OpenAI's GPT-4o-mini compensates for retrieval limitations by generating more accurate, fluent, and contextually enriched responses than the locally hosted "Llama3.2 3B Instruct" (Quantized) effectively bridging gaps in retrieval inaccuracies. Finally, this research underscores the critical role of domain adaptation techniques and persona-based retrieval augmentation in optimizing AI travel assistants with RAG architectures, paving the way for future advancements in personalized, context-aware, and highly adaptable tourism recommendation systems.

Η παρούσα διπλωματική εργασία παρουσιάζει ένα προηγμένο σύστημα παραγωγής κειμένου με ενίσχυση ανάκτησης (Retrieval-Augmented Generation, RAG), σχεδιασμένο για τη βελτίωση των συστημάτων συστάσεων που σχετίζονται με τον τουρισμό. Το προτεινόμενο σύστημα ενσωματώνει πολλαπλές σύγχρονες μεθοδολογίες ανάκτησης, όπως η Υβριδική Ανάκτηση Διανυσμάτων (Hybrid Vector Retrieval), AutoMerge, HyDE και GraphRAG, καθεμία από τις οποίες αξιολογήθηκε μέσω εκτενών πειραμάτων σε ένα συνθετικό, ειδικά προσαρμοσμένο σύνολο δεδομένων, με στόχο τη βελτίωση της ακρίβειας και της σχετικότητας των πληροφοριών που αφορούν τον τουρισμό. Εκμεταλλευόμενο τις ειδικές γνώσεις του πεδίου του τουρισμού, το σύστημα στοχεύει να παρέχει δυναμικές και εξατομικευμένες συστάσεις για τους ταξιδιώτες. Επιπλέον, η εργασία ενσωματώνει ένα κομμάτι εξατομικεύσεως που κατασκευάζει δυναμικά ένα προφίλ χρήστη μέσω της διαδικασίας σχεδιασμού και βελτίωσης προτροπών (prompt engineering), ενισχύοντας έτσι την ικανότητα του συστήματος να προσαρμόζει τις απαντήσεις βάσει ατομικών χαρακτηριστικών και προτιμήσεων, όπως η ηλικιακή ομάδα, η εθνικότητα, η ταξιδιωτική ομάδα, ο σκοπός του ταξιδιού κ.ά., προσφέροντας μια εξατομικευμένη εμπειρία ταξιδιωτικής υποστήριξης. Επιπρόσθετα, δίνεται έμφαση σε μια αυτοματοποιημένη, ανοικτού κώδικα προσέγγιση από άκρο σε άκρο (End-to-End), καθιστώντας το σύστημα προσαρμόσιμο και προσβάσιμο, για μελλοντικές έρευνες και εφαρμογές «σε πραγματικές συνθήκες», στον τομέα του τουρισμού. Για την αξιολόγηση της αποτελεσματικότητάς του, διεξήχθη εκτενής αριθμός πειραμάτων χρησιμοποιώντας ένα σύνολο δεδομένων που δημιουργήθηκε ειδικά για τον τομέα του τουρισμού και με τη βοήθεια μετρικών ποιότητας ανάκτησης και γενεσιουργίας. Σε αυτήν τη βάση, η παρούσα διπλωματική εργασία, διερευνά περαιτέρω τη σύνθετη σχέση μεταξύ των στοιχείων ανάκτησης και γενεσιουργίας, αναλύοντας πώς διάφορες στρατηγικές ανάκτησης επηρεάζουν την ακρίβεια, τη συμφραζόμενη συνοχή και την προσαρμοστικότητα των παραγόμενων απαντήσεων. Η έρευνα επεκτείνει τις διάφορες μεθοδολογίες ανάκτησης, συμπεριλαμβανομένων των διανυσματικών, υβριδικών και βασισμένων σε γράφους προσεγγίσεων, αξιολογώντας την αποτελεσματικότητά τους στον τομέα του τουρισμού και στη δυναμική προσαρμογή σε ποικίλα ερωτήματα χρηστών. Επιπλέον, η έρευνα εξετάζει τον αντίκτυπο των στρατηγικών διαχωρισμού εγγράφων (document segmentation) και των μηχανισμών συμφραζόμενης ανακατάταξης (context re-ranking) στην ενίσχυση της σχετικότητας των ανακτημένων πληροφοριών, ενώ παράλληλα ελαχιστοποιεί τις ψευδαισθήσεις (hallucinations) στα παραγόμενα αποτελέσματα. Μέσω της συστηματικής αξιολόγησης αυτών των προσεγγίσεων, το πλαίσιο στοχεύει στην καθιέρωση πρακτικών για τη βελτιστοποίηση της ανάκτησης στις εφαρμογές τουρισμού που στηρίζονται από τη τεχνητή νοημοσύνη. Τα αποτελέσματα της έρευνας δείχνουν ξεκάθαρα ότι η αρχιτεκτονική HyDE υπερέρχει έναντι των άλλων, τόσο για μικρά σε υπολογιστικό κόστος μοντέλα ανοιχτού κώδικα που φιλοξενούνται τοπικά (Llama3.2 3B Instruct, BGE-M3), όσο και για μοντέλα που παρέχονται από εξωτερικούς παρόχους Μεγάλων Γλωσσικών Μοντέλων (MGΜ) όπως η OpenAI (GPT-4o-mini, text-embedding-3-small). Επίσης, τα πειράματα που συγκρίνουν τα τοπικά μοντέλα με αυτά της OpenAI για την προσέγγιση HyDE αποκαλύπτουν, ότι το

τοπικά φιλοξενούμενο BGE-M3 υπερτερεί του μοντέλου text-embedding-3-small στην ανάκτηση πληροφορίας, ενώ το GPT-4o-mini της OpenAI αντισταθμίζει τους περιορισμούς στην ανάκτηση παράγοντας πιο ακριβείς, ρέουσες και συμφραζόμενα εμπλουτισμένες απαντήσεις από ό,τι το τοπικό “Llama3.2 3B Instruct” (Quantized), γεφυρώνοντας αποτελεσματικά τις ελλείψεις που προκύπτουν από τις ανακρίβειες στην ανάκτηση. Πέραν των ποσοτικών συγκρίσεων επιδόσεων, η παρούσα έρευνα διεξάγει μια λεπτομερή ποιοτική ανάλυση των απαντήσεων που εμπλουτίζονται με ανάκτηση, εστιάζοντας στην παραγματολογική ακρίβεια (factual correctness) και στην προσαρμοστικότητα του συστήματος σε ποικίλες καταστάσεις συζητήσεων με χρήστες. Εξετάζοντας τη σχέση μεταξύ της ανάκλησης (recall) στην ανάκτηση και της ποιότητας της γενεσιουργίας, η μελέτη αποδεικνύει ότι οι αρχιτεκτονικές με υψηλότερα ποσοστά ανάκλησης (recall), όπως η HyDE, επιτρέπουν την παραγωγή απαντήσεων που είναι περισσότερο τεκμηριωμένες και σχετικές με τα συμφραζόμενα, ενώ τα μοντέλα με χαμηλότερη την μετρική αυτή, όπως το AutoMerge, δυσκολεύονται να διατηρήσουν τη συνοχή των απαντήσεων. Το προτεινόμενο σύστημα έχει σχεδιαστεί να είναι αρθρωτό (modular) και επεκτάσιμο, διευκολύνοντας την απρόσκοπτη ενσωμάτωση εξωτερικών πηγών γνώσης και επιτρέποντας ενημερώσεις σε πραγματικό χρόνο, ώστε οι απαντήσεις του να προσαρμόζονται δυναμικά στις μεταβαλλόμενες πληροφορίες που αφορούν τον τουρισμό. Αυτό αντιμετωπίζει άμεσα τους περιορισμούς των στατικών συνόλων δεδομένων, εξασφαλίζοντας ότι οι ταξιδιωτικές συστάσεις που βασίζονται στην τεχνητή νοημοσύνη παραμένουν ακριβείς και ενημερωμένες. Τέλος, τα ευρήματα υπογραμμίζουν τη σημαντικότητα των τεχνικών προσαρμογής στον συγκεκριμένο τομέα, ως προς τη βελτιστοποίηση των συστημάτων συστάσεων που βασίζονται σε ενίσχυση ανάκτησης (RAG), ανοίγοντας τον δρόμο για περαιτέρω προόδους στην υποβοήθηση του ταξιδιωτικού τομέα με τη βοήθεια της τεχνητής νοημοσύνης. Μερικές μελλοντικές κατευθύνσεις αυτής της διπλωματικής εργασίας μπορούν να περιλαμβάνουν την εξερεύνηση προσαρμοστικών μηχανισμών ανάκτησης (adaptive retrieval mechanisms) που ρυθμίζουν δυναμικά τις στρατηγικές ανάκτησης βάσει της πολυπλοκότητας του ερωτήματος του χρήστη, εξασφαλίζοντας πιο ακριβείς συστάσεις. Επιπλέον, μπορεί να εξεταστεί η περεταίρω εκπαίδευση (fine-tuning) των μοντέλων ανάκτησης και γενεσιουργίας σε δεδομένα ειδικά για το πεδίο του τουρισμού, με στόχο τη βελτίωση των απαντήσεων σε εξειδικευμένες εφαρμογές τουρισμού. Μπορεί επίσης να γίνει επέκταση του συστήματος ώστε να υποστηρίζει πολυγλωσσική και δια-τομεακή (π.χ. τομέα υγείας) λειτουργικότητα, έτσι ώστε να βελτιώσει την προσβασιμότητα για τους ταξιδιώτες. Επιπλέον, μια μελλοντική κατεύθυνση είναι η χρήση του συστήματος σε “πραγματικές συνθήκες” και αξιολόγηση του από τους χρήστες για την εκτίμηση της απόδοσης σε πρακτικά σενάρια, συγκεντρώνοντας ανατροφοδότηση (feedback). Αυτές οι εξελίξεις έχουν τη δυνατότητα να βελτιώσουν την εμπειρία υποστήριξης και υποβοήθησης χρηστών με τη χρήση τεχνητής νοημοσύνης, καθιστώντας τα συστήματα συστάσεων όχι μόνο πιο ακριβή, αλλά και πιο προσαρμοστικά, κλιμακούμενα και προσανατολισμένα στον χρήστη.

Acknowledgements

Writing this thesis was not an easy task and, I would like to express my deepest gratitude to my supervisor, Dr. Athena Vakali, for her guidance, feedback, and support throughout this journey. Her expertise has been invaluable in shaping this thesis.

I am equally grateful to my co-supervisor, Dr. Ilias Dimitriadis, whose constructive criticism, technical advice, and patient mentoring have greatly enriched this work. Their willingness to share their knowledge and experience has made a significant impact on both my research and more importantly my personal development.

I also wish to thank my family, partner and friends for their endless encouragement and support during the challenging times of this academic endeavor. Finally, I appreciate the contributions of everyone who supported me directly or indirectly, helping me to achieve this milestone.

Contents

Declaration of Authorship	i
Abstract	iii
Acknowledgements	vi
List of Figures	x
List of Tables	xi
Abbreviations	xii
1 Introduction	1
1.1 Overview	2
1.1.1 Rise Of AI	3
1.1.2 Challenges of AI	5
1.1.3 AI Under the RAG	6
1.2 Idea and Motivation	8
1.3 Contribution	9
1.4 Thesis Structure	11
2 Literature Review	12
2.1 RAG Methodologies, Benefits, and Related Work	12
2.2 Vector Stores: Approaches, Types of Retrievers, and Graph Knowledge Retrieval	13
2.3 Query Prompting Optimization Methodologies	15
2.4 Retrieval and LLM Architectures for Generation	17
2.5 Evaluation Methods	19
2.6 LLM based Application on the Tourism Domain	21
3 RAG Fundamentals	24
3.1 Principles and Fundamentals	24
3.1.1 Retrieval	24
3.1.1.1 Sparse Retrieval	25
3.1.1.2 Dense Retrieval	26
3.1.2 Pre-Post Processing Enhancement	27

3.1.2.1	Pre-Retrieval Enhancement	27
3.1.2.2	Post-Retrieval Enhancement	28
3.1.3	Evaluating RAG Architectures	29
4	TouaRAG Tools & System Architecture	31
4.1	Development Libraries and Frameworks	31
4.1.1	Llamaindex	31
4.1.2	Hugging Face API	32
4.1.3	OpenAI API	32
4.1.4	PostgreSQL with PGVector	32
4.1.5	RAGAS	33
4.1.6	GraphRAG	33
4.2	General Architecture of System	33
4.3	Synthetic Test Data Generation	35
5	TouaRAG Core Components & Implementation	38
5.1	Framework Components	38
5.1.1	Data Generation	38
5.1.2	Configuration and Utilization	41
5.1.3	Prompt Engineering and Personalization	42
5.1.4	LLM Querying	45
5.1.5	Evaluation	46
5.2	Implemented RAG Architectures	47
5.2.1	Baseline	47
5.2.2	Hybrid Vectors	49
5.2.3	AutoMerge	51
5.2.4	HyDE	52
5.2.5	Local GraphRAG	54
6	Experimentation & Validation	56
6.1	Overview	56
6.2	Experimental Setup	57
6.2.1	Dataset	57
6.2.2	Computational Environment	58
6.3	Evaluation Metrics	58
6.3.1	Retrieval Quality Metrics	59
6.3.1.1	Context Precision	59
6.3.1.2	Context Recall	61
6.3.1.3	Context Entity Recall	62
6.3.2	Generative Quality Metrics	63
6.3.2.1	Response Relevancy	64
6.3.2.2	Faithfulness	65
6.3.2.3	Semantic Similarity	67
6.3.2.4	Factual Correctness	68
6.4	Results	71
6.4.1	Local Models Experimentation	71
6.4.2	OpenAI Models Experimentation	74

6.4.3	Best Architecture per Model Comparison	77
7	Conclusions & Future Work	80
7.1	Retrieval and Generative Performance Trade-offs	80
7.2	Local vs. OpenAI Model Comparisons	81
7.3	Future Work	82
7.4	Final Remarks	83
	 Bibliography	 84

List of Figures

1.1	Projected Job loss and Growth for 2028 vs 2018. Medium, The Research Challenges of the AI Labor Market Challenges, Golub Capital Social Impact Lab @ Stanford GSB	3
1.2	Enterprises using AI technologies, EU, 2023 and 2024, Eurostat	4
1.3	GitHub Stars trend of Haystack framework. From Star History Monthly, "RAG Frameworks", August 2024.	7
1.4	Retrieval Augmented Generation Market Size. From Grand View Research, Retrieval Augmented Generation Market Size, Share & Trends Analysis Report By Function, By Application, By Deployment (Cloud, On-premises), By End-use (Healthcare, Education), By Region, And Segment Forecasts, 2024 - 2030. . . .	8
2.1	RAG Architecture Overview (Retrieval-Augmented Generation for AI-Generated Content: A Survey, Zhao et al., 2024 [37]).	12
2.2	Communities in GraphRAG during summarization step (From Local to Global: A Graph RAG Approach to Query-Focused Summarization, Edge et al., 2024). [7]	19
4.1	Knowledge Graph Creation Workflow for Synthetic Data Generation in RAGAS. (RAGAS: Automated Evaluation of Retrieval Augmented Generation, Es et al., 2023).	37
5.1	Baseline Architecture Workflow Diagram	49
5.2	Hybrid Vectors Architecture Workflow Diagram	51
5.3	AutoMerge Architecture Workflow Diagram	52
5.4	HyDE Architecture Workflow Diagram	54

List of Tables

2.1	Categorization of domain-specific papers based on their contributions to Information Augmentation (IA), Multiple Retrieval Methodologies (MRM), LLM Generation (LG), RAG, and Persona Integration (PI).	23
6.1	Model Configurations	58
6.2	Evaluation Results Across Different RAG Architectures with locally loaded models (Llama3.2 3B Instruct Quantized and BGE-M3)	73
6.3	Evaluation Results Across Different RAG Architectures with OpenAI models (GPT4o-mini and text-embedding-3-small). Retrieval Quality metrics in GraphRAG not applicable.	77
6.4	Comparison of HyDE Local and HyDE OpenAI across Retrieval and Generative Metrics.	78

Abbreviations

AI	Artificial Intelligence
ANN	Approximate Nearest Neighbor
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BLEU	Bilingual Evaluation Understudy
BM25	Best Matching 25
CAGR	Compound Annual Growth Rate
CL	Computational Linguistics
CNN	Convolutional Neural Network
COT	Chain Of Thought
CPU	Central Processing Unit
DiskANN	Disk-based Approximate Nearest Neighbor
DPR	Dense Passage Retrieval
EU	European Union
FLARE	Forward Looking Active Retrieval
FN	False Negatives
FP	False Positives
GDP	Gross Domestic Product
GPT	Generative Pre-Trained Transformer
GPU	Graphics Processing Unit
HNSW	Hierarchical Navigable Small-World Network
HMANN	Hierarchical Memory-based Approximate Nearest Neighbor
HyDE	Hypothetical Document Embeddings
IA	Information Augmentation
ICL	In Context Learning

IR	I nformation R etrieval
KG	K nowledge G raph
LG	L LM G eneration
LLM	L arge L anguage M odel
LoRA	L ow- R ank A daptation
LTS	L ong T erm S upport
METEOR	M etric for E valuation of T ranslation with E xplicit O rdering
MRM	M ultiple R etrieval M ethodologies
MRR	M ean R eciprocal R ank
NER	N amed E ntity R ecognition
NLG	N atural L anguage G eneration
NLP	N atural L anguage P rocessing
NLU	N atural L anguage U nderstanding
ORPO	O dds R atio P reference O ptimization
OS	O perating S ystem
PI	P ersona I ntegration
QA	Q uestion A nswering
R2G	R etrieve- R erank- G enerate
RAG	R etrieval A ugmented G eneration
RAM	R andom A ccess M emory
RAPTOR	R ecursive A bstractive P rocessing for T ree- O rganized R etrieval
RECOMP	R etrieve, C ompress, P repend
RNN	R ecurrent N eural N etwork
RoBERTa	R obustly O ptimized B ERT P retraining A pproach
ROUGE	R ecall- O riented U nderstudy for G isting E valuation
RRF	R eciprocal R ank F usion
SEQ2SEQ	S equence-to- S equence
SFT	S upervised F ine- T uning
SUS	S oftware U sability S cale
TAM	T echnology A cceptance M odel
TF-IDF	T erm F requency- I nverse D ocument F requency
TP	T rue P ositives
UGC	U ser- G enerated C ontent

USD **United States Dollar**

Chapter 1

Introduction

Artificial intelligence (AI) has evolved from a speculative concept into a transformative technology reshaping every facet of modern society. In its earliest days, AI was confined to rule-based systems and symbolic reasoning. Today, advanced techniques, ranging from deep learning to reinforcement learning, fuel breakthroughs in natural language processing (NLP) and computer vision. The rapid development of Large Language Models (LLMs), exemplified by ChatGPT and similar systems, has not only sparked widespread enthusiasm to the public, but has also brought forward profound challenges.

Modern AI applications have become so ubiquitous that many people interact with them daily, often without even realizing it. Voice assistants like Siri, Alexa, and Google Assistant help schedule appointments and control smart home devices, recommendation engines on streaming platforms and e-commerce sites curate content and product suggestions and navigation apps, powered by real-time data analytics, optimize our travel routes. Moreover, AI-driven applications in healthcare assist in diagnostics, while personalized learning systems tailor educational experiences to individual needs. In finance, algorithmic trading and risk assessment models support more efficient decision-making, illustrating AI's broad potential to enhance productivity and improve quality of life. At the same time, the transformative power of AI has introduced new challenges. For example, while LLMs like ChatGPT have unlocked unprecedented capabilities in generating human-like text. But these models can also "hallucinate", producing plausible sounding but inaccurate or outdated information—raising concerns about reliability and trustworthiness.

One critical challenge in generative AI is the phenomenon of hallucination, where models generate fluent, plausible-sounding text that is factually incorrect or misleading. These hallucinations can range from minor inaccuracies to completely fabricated information, making them particularly problematic in domains where factual correctness is essential, such as healthcare, law, and finance. Recent studies have categorized hallucinations into different types, such as

intrinsic hallucinations, where the model fabricates information due to internal biases, and extrinsic hallucinations, where the generated content is not supported by retrieved or factual evidence [10, 12]. Addressing this issue remains a major focus of AI research, as ensuring the reliability and trustworthiness of AI-generated responses is critical for their adoption in real-world applications.

The rest of this chapter includes an overview of the current landscape of AI in relation to Chatbot Assistants and LLM models, a brief description of such system approaches, the idea and motivation behind this thesis, the contribution of this work and an outline of the overall thesis.

1.1 Overview

The recent surge in AI research and development has been driven by a combination of technological advances and substantial investments. With billions of dollars being funneled into AI research globally, innovations in deep learning have resulted in LLMs that can perform tasks previously thought exclusive to human cognition, from drafting emails even to composing poetry. Subsequently the positive impacts of AI are quite significant. AI-driven automation has dramatically increased productivity across numerous sectors by streamlining repetitive tasks and enabling human workers to focus on creative and strategic endeavors. For example, many tech start-ups report that integrating AI tools into their workflows has not only boosted efficiency but also reallocated human resources toward innovation and the creation of new roles¹. In healthcare, AI applications assist in rapid and accurate diagnostics, while in finance, they enhance fraud detection and risk management. Additionally, AI-powered recommendation systems have transformed digital experiences by providing personalized content that enriches user engagement. These advances contribute to economic growth, foster new business opportunities, and ultimately improve the quality of life by enabling more informed decision-making and better services.

¹The Sunday Times: AI tools do not cost jobs – they boost productivity, say start-ups
<https://www.thetimes.com/business-money/entrepreneurs/article/ai-wont-cut-jobs-say-tech-start-ups-enterprise-network-f2wdrkcgr>

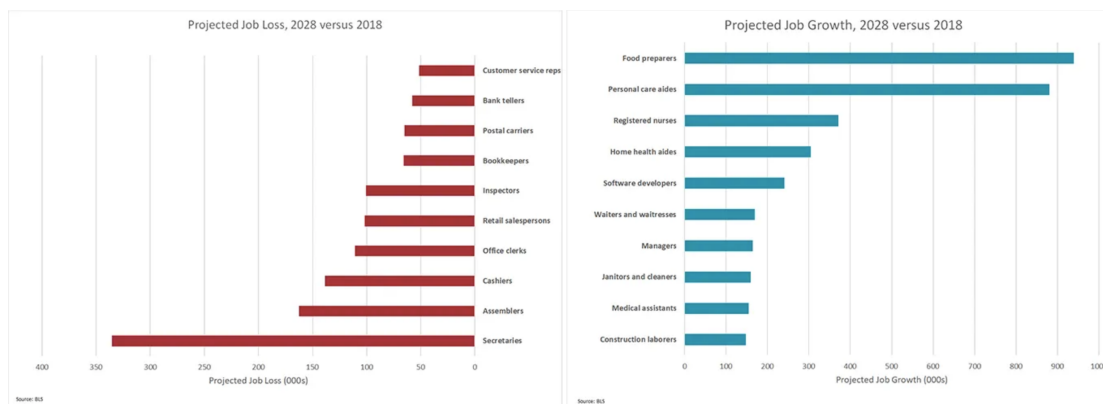


FIGURE 1.1: Projected Job loss and Growth for 2028 vs 2018. Medium, The Research Challenges of the AI Labor Market Challenges, Golub Capital Social Impact Lab @ Stanford GSB

However, alongside these capabilities comes a growing body of evidence suggesting that AI could displace human labor. Projections from various studies [1] indicate that millions of jobs might be at risk as routine procedures and even some complex tasks become automatable in a satisfactory manner. The socioeconomic implications of such shifts are profound, fueling debates about job displacement, the future of work, and the need for robust social policies to manage transitions in the labor market ². These shifts have ignited discussions among economists and industry leaders, urging policymakers to address potential disruptions before they become irreversible.

Moreover, AI's capabilities have extended into the realm of information dissemination. The very strength of modern large language models (LLMs), their ability to generate fluent, "human like" text, can be misused to produce convincing yet misleading narratives. This has given rise to phenomena like fake news and misinformation, where AI-generated content can be weaponized to manipulate public opinion and exacerbate social divisions.

1.1.1 Rise Of AI

The evolution of artificial intelligence (AI) is a story of continuous innovation and transformation that spans over seven decades. In its nascent stages during the 1950s and 1960s, AI research was primarily dominated by symbolic reasoning and rule-based systems. Pioneers like Alan Turing, Marvin Minsky, and John McCarthy laid the theoretical foundations of AI, sparking early optimism about machines that could mimic human thought. However, early ambitions were tempered by practical challenges—including limited computational power and insufficient data—which led to periods of stagnation commonly referred to as "AI winters."

²Medium: The Research Challenges of the AI Labor Market Challenges, Golub Capital Social Impact Lab @ Stanford GSB https://medium.com/@gsb_silab/the-research-challenges-of-the-ai-labor-market-challenges-272f69d98f9d

The resurgence of AI in the early 2000s can largely be attributed to the advent of machine learning and, more critically, deep learning. As computational resources expanded and large datasets became more accessible, researchers began to develop neural network architectures capable of learning complex patterns directly from data. Breakthroughs in convolutional neural networks (CNNs) revolutionized computer vision, while advancements in recurrent neural networks (RNNs) and, later, transformer-based models dramatically improved natural language processing (NLP) tasks. These deep learning methods enabled AI systems to achieve levels of performance that were once thought unattainable, setting the stage for the emergence of LLMs. Over the past few years, the rapid development and deployment of large language models (LLMs) have captured public attention and transformed the way we interact with technology. Today, AI applications are deeply integrated into everyday life: voice assistants like Siri, Alexa, and Google Assistant help manage our schedules, recommendation engines tailor our entertainment and shopping experiences and navigation apps optimize our travel routes in real time. More and more companies are trying to integrate AI systems or procedures into their pipelines as reported by Eurostat where even the percentage is quite uplifted from 2023 to 2024 in terms of AI adoption ³.

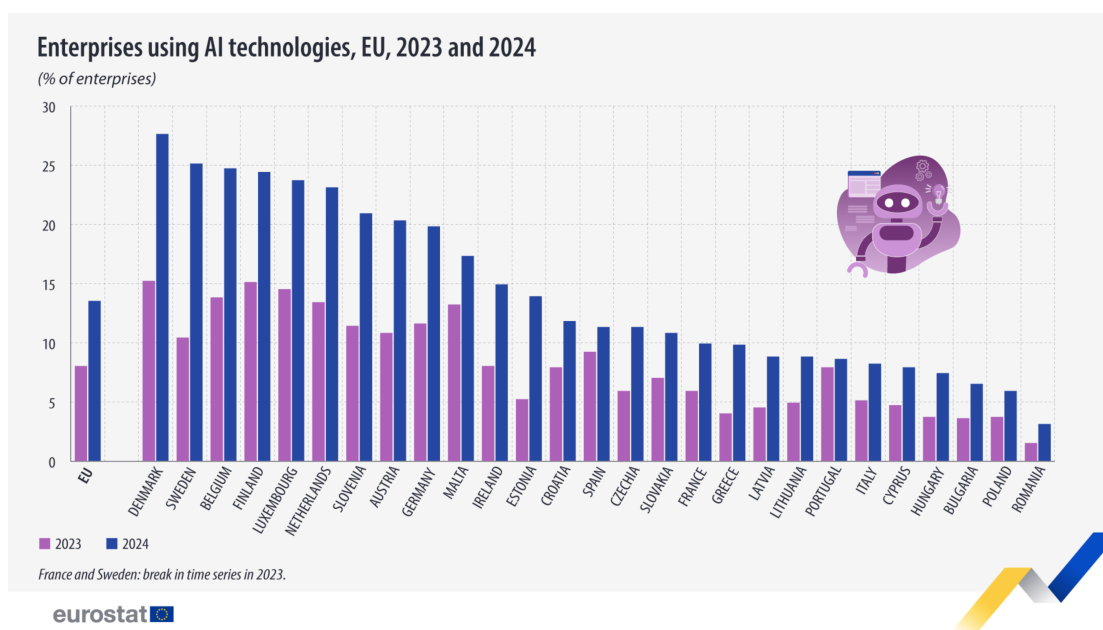


FIGURE 1.2: Enterprises using AI technologies, EU, 2023 and 2024, Eurostat

In healthcare, AI-driven diagnostic tools aid physicians in early disease detection, while in finance, sophisticated algorithms support fraud detection and risk assessment. These advancements not only enhance productivity but also open up new avenues for innovation and economic growth. Moreover, the tourism sector is beginning to harness these technologies—using AI-powered systems to generate personalized travel recommendations and optimize itinerary

³Source: Eurostat - Usage of AI technologies increasing in EU enterprises, <https://ec.europa.eu/eurostat/web/products-eurostat-news/w/ddn-20250123-3>

planning by integrating real-time data from diverse sources. This growing interest in AI-driven solutions across industries sets the stage for exploring advanced architectures, such as Retrieval-Augmented Generation (RAG), which this thesis focuses on within the context of tourism.

1.1.2 Challenges of AI

Despite the transformative potential of artificial intelligence, its rapid advancement has brought several critical challenges that must be addressed. One of the foremost issues is the problem of hallucination, a phenomenon where LLMs generate fluent, human-like text that is, however, factually inaccurate to the original content or outdated [10, 12]. These hallucinations undermine the reliability and trustworthiness of AI systems, especially in applications where precision is necessary, such as healthcare and finance.

Another significant challenge is the dependency on massive datasets that quickly become obsolete. The training data for LLMs are often static, meaning that as new information emerges, the models may continue to rely on outdated or incomplete knowledge. As discussed by Cheng et al. [6], they investigate the temporal alignment of LLMs' training data, introducing the concept of an "effective cutoff" to assess how current the knowledge within these models is. This temporal gap can lead to errors in applications requiring real-time information, a problem that is particularly acute in dynamic sectors like tourism where updated travel data and local insights are essential for effective service delivery.

Bias and fairness also remain persistent issues. Since AI systems learn from historical data, they can inadvertently replicate and amplify existing biases, leading to discriminatory outcomes in automated decision-making. This not only poses ethical challenges but can also result in significant socioeconomic disparities—fueling debates about job displacement and the broader impact on labor markets.

Moreover, the high computational and financial costs associated with training and maintaining advanced AI models limit their accessibility and scalability. The energy consumption and environmental footprint of these systems are additional concerns that must be balanced against their economic benefits.

Finally, the ability of AI to generate persuasive narratives has made it a potent tool for misinformation. The misuse of generative AI for creating fake news or manipulating public opinion threatens to exacerbate social divisions and erode trust in digital information sources.

These challenges—ranging from technical limitations like hallucination and data staleness to broader ethical and socioeconomic concerns—highlight the urgent need for new architectural paradigms. In response, advanced frameworks such as Retrieval-Augmented Generation (RAG)

have emerged. By integrating robust retrieval mechanisms that reference up-to-date, verifiable data, RAG architectures offer a promising approach to mitigating many of the shortcomings of traditional generative models. This hybrid strategy is especially relevant in high-stakes domains, including tourism, where ensuring the accuracy of information can significantly enhance user experience and decision-making. In this industry, where accurate and timely information directly influences user satisfaction and decision-making, even minor errors can have significant consequences. The global tourism market, for instance, is a multitrillion-dollar industry with profound economic implications. In 2024, the travel and tourism industry's contribution to global GDP is projected to reach \$11.1 trillion—making up 10% of global GDP—and supporting nearly 348 million jobs worldwide ⁴. Such figures highlight the enormous economic stakes involved and the potential benefits of AI systems that can deliver reliable, context-aware information.

1.1.3 AI Under the RAG

Retrieval-Augmented Generation (RAG) represents a significant evolution in AI architectures, specifically designed to address many of the challenges inherent in traditional generative models. At its core, RAG combines two complementary components, a generation model (most likely LLM) that produces fluent, human-like text and a retrieval system that fetches up-to-date, relevant external data. This integration enables the system to ground its outputs in verifiable sources, effectively mitigating issues such as hallucination and data staleness.

Traditional LLMs are typically trained on static datasets, which can quickly become outdated. As a result, these models might generate plausible-sounding yet inaccurate or incomplete information. RAG architectures overcome this limitation by incorporating a dynamic retrieval module. When a query is processed, the system first retrieves pertinent documents or data fragments from a curated knowledge base and then conditions the generative model on this retrieved context. The outcome is a response that is not only coherent and contextually rich but also anchored in current and reliable information.

Moreover, the retrieval component helps reduce inherent biases by enabling the system to reference a diverse set of up-to-date sources rather than relying solely on historical training data. This modular approach also offers flexibility in terms of implementation. RAG systems can be tailored in various forms—such as sequence-level, token-level, or hybrid models—to suit the specific requirements of different applications. For instance, in industries like healthcare or finance, where precision is critical, a RAG system can be fine-tuned with domain-specific data like tourism related ones, thus improving accuracy and reducing error rates.

⁴Reuters: Travel industry to contribute record \$11 trillion to global GDP in 2024. [Link](#)

Finally, the rising adoption of RAG frameworks is evident in the growing number of open-source projects and research initiatives centered around this paradigm. As highlighted in recent analyses of RAG tool adoption⁵, frameworks such as Haystack⁶, see Figure 1.3 below, have witnessed substantial increases in GitHub stars over the past year. According to the latest statistics, Haystack has grown by +10k stars from 2022 until August of 2024.

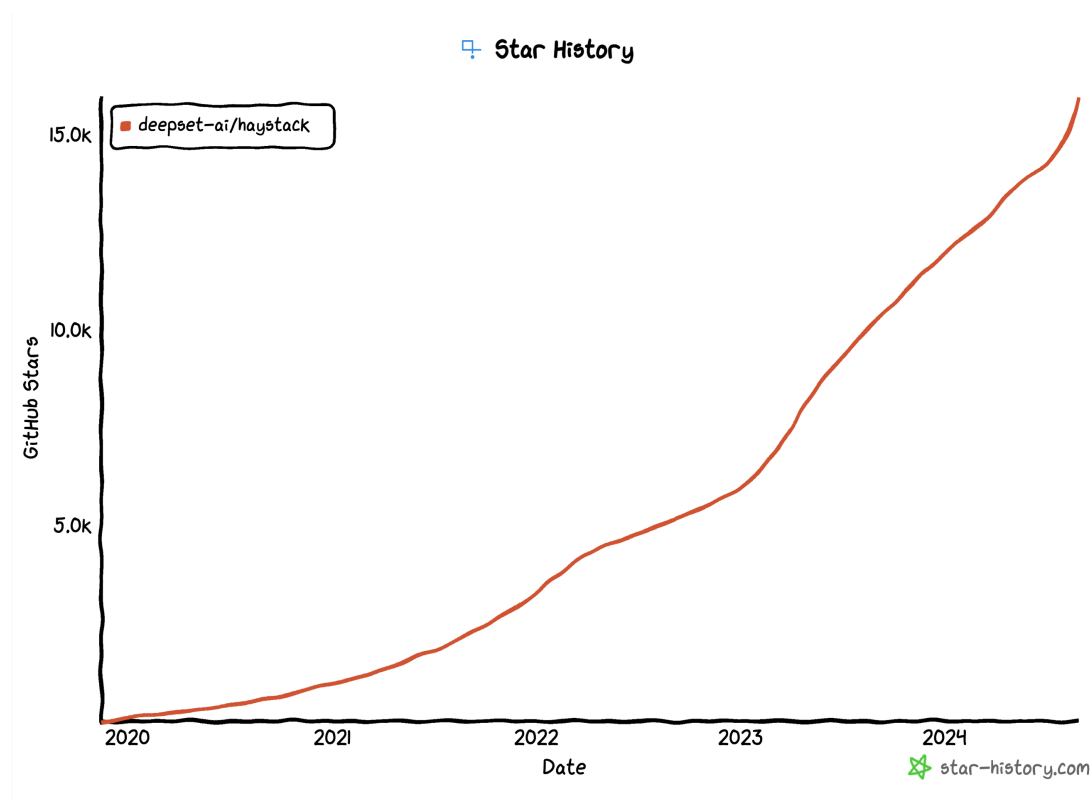


FIGURE 1.3: GitHub Stars trend of Haystack framework. From Star History Monthly, "RAG Frameworks", August 2024.

This substantial increase has also been depicted in reports with respect to the market size of RAGs. As presented in Figure 1.4, in 2023, the retrieval-augmented generation market was valued at around USD 1,042.7 million and is anticipated to grow at a Compound annual growth rate (CAGR) of 44.7% between 2024 and 2030⁷.

⁵Star History Monthly, "RAG Frameworks", August 2024. Available at: <https://star-history.com/blog/rag-frameworks>

⁶Haystack GitHub repository: <https://github.com/deepset-ai/haystack>

⁷Grand View Research, Retrieval Augmented Generation Market Size, Share & Trends Analysis Report By Function, By Application, By Deployment (Cloud, On-premises), By End-use (Healthcare, Education), By Region, And Segment Forecasts, 2024. Available at: <https://www.grandviewresearch.com/industry-analysis/retrieval-augmented-generation-rag-market-report>

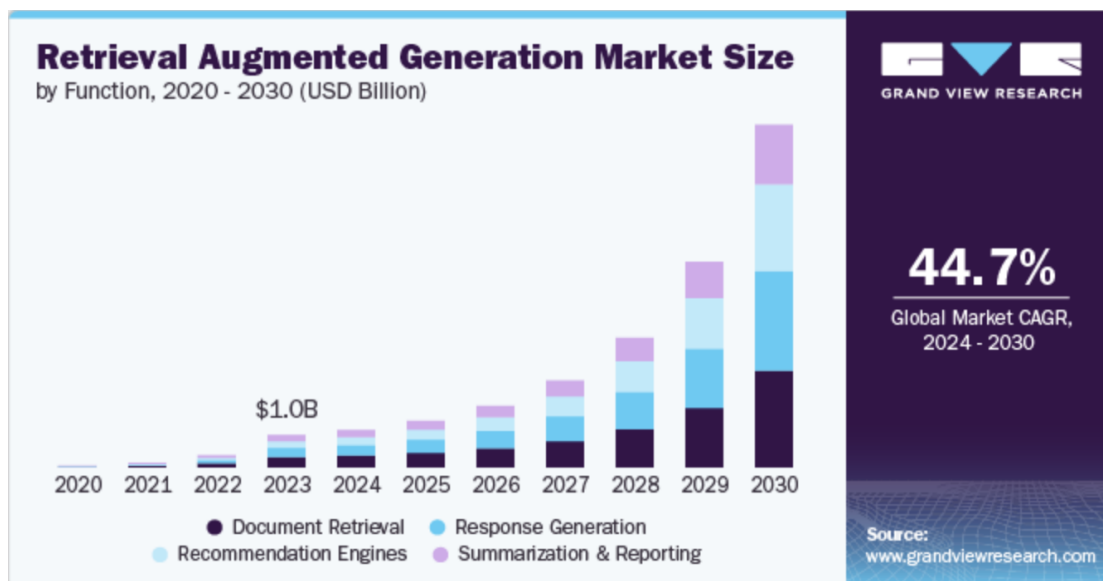


FIGURE 1.4: Retrieval Augmented Generation Market Size. From Grand View Research, Retrieval Augmented Generation Market Size, Share & Trends Analysis Report By Function, By Application, By Deployment (Cloud, On-premises), By End-use (Healthcare, Education), By Region, And Segment Forecasts, 2024 - 2030.

This surge in interest is driven by the need for AI systems that can dynamically integrate external knowledge while maintaining high standards of factual accuracy. With organizations across diverse sectors exploring RAG-powered applications, the field is poised for continued expansion, promising even more refined and adaptable AI solutions in the near future.

1.2 Idea and Motivation

The tourism industry stands out as a uniquely challenging yet highly rewarding domain for the application of advanced AI technologies. Unlike more static domains, tourism is characterized by rapidly changing variables: weather conditions, local events, travel advisories, dynamic pricing, and evolving consumer preferences. A key limitation here is the **overwhelming volume of information** travelers encounter from countless sources such as online reviews, social media posts, travel blogs, and official tourism websites. This often makes the decision-making process both overwhelming and time-consuming. Moreover, **conflicting and unreliable information** exacerbates the confusion, as there is no direct way of aggregating diverse data into a single, user-friendly interface that travelers can interact with.

This thesis is motivated by the need to harness Retrieval-Augmented Generation (RAG) architectures to provide accurate, context-aware, and timely travel recommendations that can significantly enhance user experience in this dynamic environment. More specifically, the problem of excessive information overload persists for tourists who rely on social media, travel

blogs, official tourism websites, and real-time local advisories. As a result, travelers often struggle when selecting a destination or planning an itinerary. By leveraging RAG architectures, the main motivation is to deliver recommendations that are not only tailored to individual user profiles but also updated in response to evolving trends and real-time data according to provided external knowledge while, they automatically digest and process various different sources. Addressing this problem has the potential to simplify decision making for travelers and enhance their overall experience.

A significant limitation with generative AI models as also mentioned in previous section, is their **tendency to "hallucinate"**, thus producing fluent but inaccurate responses. In high-stakes applications such as tourism planning, even minor inaccuracies can lead to traveler dissatisfaction or misinformed decisions. RAG offers a promising solution by anchoring the generation process with verifiable, external data, thereby reducing the risk of hallucinations. This research is motivated by the desire to create a system that not only generates human-like text but does so with a high degree of factual reliability. By incorporating robust retrieval mechanisms into the generation pipeline, we aim to enhance the trustworthiness of AI-driven travel recommendations and establish best practices for deploying such systems in real-world settings.

Furthermore, tourism is inherently local and time-sensitive. Factors such as sudden weather changes, local festivals, temporary travel restrictions, and unexpected events can greatly affect the travel experience. Another critical limitation arises from traditional LLMs, which are typically **trained on static datasets** and are often **unable to reflect these rapid changes**. RAG architectures mitigate this by integrating external data retrieval mechanisms. This approach could enable the system to fetch the most current information from curated knowledge bases and online sources ensuring that recommendations are grounded in the latest available data. The current work of this thesis, can be extended in the future with the appropriate APIs to fetch data from live resources and be integrated within the knowledge base that the RAG architecture is using. Our motivation is to harness this dynamic integration to bridge the gap between static pre-trained models and the real-world variability inherent in tourism.

1.3 Contribution

To this end we propose **TouaRAG**, an open source RAG framework focused on the tourism domain which attempts to bridge the gap and address the discussed challenges. **TouaRAG** makes specific contributions to the field of AI applications in tourism through the use of RAG architectures which are primarily based to the above motivation and ideas of Section 1.2. The proposed framework contributes to the field of RAG architectures for enhancing tourism application, in the ways described below:

1. **Benchmarking State-of-the-Art RAG Architectures:** We systematically evaluate and benchmark a range of cutting-edge RAG architectures, both from open-source, industry-leading platform libraries and State of The Art approaches in past literature. In this thesis, extensive evaluation of five distinct retrieval-augmented generation architectures was performed using domain-specific tourism queries as the testbed. The analysis spans both local and commercially available models to provide more depth into the benchmarking results. This evaluation aims to pinpoint strengths and weaknesses of each architecture in terms of semantic relevance, factual correctness and a variety of other indicators with tourism domain related data, thereby providing actionable insights for deploying and optimizing RAG systems in specialized applications. This step **directly addresses the problem of hallucination in generative AI** by identifying the most reliable models and configurations under tourism-specific conditions.
2. **Domain-specific Evaluation and Dataset Creation:** We include a comprehensive evaluation framework tailored to the tourism domain by leveraging the RAGAS library to **automate the synthetic data generation and evaluation pipeline**. Specifically, we construct a knowledge graph from a tourism guide about Thessaloniki and enrich it through automated transformations, to capture the nuanced semantic relationships within the domain. The use of single-hop and multi-hop query types was employed, as well as by enabling persona-based scenario synthesis via prompt engineering techniques. This structured pipeline provided a diverse and realistic dataset that rigorously tests the retrieval and response generation capabilities of our proposed RAG framework. Crucially, this dataset can be updated to reflect up-to-date conditions, thereby **addressing the limitation of static datasets**. This ensures that, those rapid changes are incorporated into the evaluation pipeline, thus having representative and realistic metrics extracted.
3. **Development of an Open Source End-to-End RAG Framework:** As part of this thesis, we design and implement an open-source RAG framework optimized for domain specific applications. The framework integrates multiple retrieval methodologies, including sparse retrieval, dense retrieval, hybrid fusion techniques, and graph-based retrieval through knowledge graphs. Furthermore, we introduce optimizations such as query augmentation, metadata enrichment, and other mechanisms to improve retrieval and factual accuracy thus reducing the hallucination effects of LLMs which are inherent. Finally an automated evaluation system is integrated to perform monitoring of the framework on-the-fly. By offering a unified end-to-end system, we **address the problem of overwhelming information**, allowing travelers to interact with a single platform that consolidates and analyzes diverse data sources, thereby enhancing their overall travel experience. The framework provides a modular and extensible structure, allowing for adaptation to different domain specific needs and in our case in the tourism domain,

capabilities such as itinerary recommendations, cultural site descriptions, and travel planning assistance. We open source our code in the following open GitHub repository: <https://github.com/DataLab-AUTH/TouaRAG>, aiming to provide a practical and scalable foundation for future research and applications in AI-driven tourism recommendations.

To sum up, the goal of this thesis is to provide an advanced Retrieval-Augmented Generation (RAG) framework for tourism applications, TouaRAG, that enhances the accuracy and relevance of travel recommendations. Moreover, it aims to systematically evaluate RAG pipelines by incorporating domain-specific knowledge, which is critical for assessing the effectiveness and reliability of such systems. Finally, this work underscores the importance of an end-to-end automated framework that is open-source, ensuring accessibility, reproducibility, and adaptability for future research and real-world applications in AI driven tourism recommendations.

1.4 Thesis Structure

The thesis is structured as follows: Chapter 1 introduces the research topic, providing an overview of the overall AI landscape and the role of RAG in it, particularly in the tourism domain. It presents the motivation and objectives of the study, outlines the key challenges in AI systems, and highlights the contributions of this work. Chapter 2 presents an overview of existing research on Large Language Models, RAG architectures and retrieval techniques, their benefits, and their application in various domains, with a focus on tourism. Chapter 3 introduces the fundamental principles behind RAG architectures, covering retrieval methods, enhancements, and strategies for evaluating RAG-based systems. Chapter 4 details the development environment, libraries, and frameworks used to build the RAG system for tourism applications. Furthermore, it outlines the general system architecture and synthetic test data generation strategies. Chapter 5 provides a detailed breakdown of the implemented RAG architectures, covering their core components and the various RAG architectures that this framework incorporates. Chapter 6 presents the experimental setup used to evaluate the RAG system, including dataset descriptions, computational environment details, and evaluation metrics. It compares the performance of different RAG architectures and the results are analyzed to provide insights into the effectiveness and limitations of each approach. Chapter 7 concludes the thesis by summarizing the key findings, highlighting the contributions made to RAG systems in the tourism domain. It discusses potential avenues for future research, including real-time data integration, domain adaptation techniques, and improvements in retrieval efficiency.

Chapter 2

Literature Review

2.1 RAG Methodologies, Benefits, and Related Work

Retrieval-Augmented Generation (RAG) is a hybrid architecture that enhances large language models (LLMs) by augmenting their generative capabilities with external retrieval mechanisms. RAG integrates two key processes, the first is the retrieval of relevant documents, passages or even graphs from external knowledge bases and the second one is the generation of responses using both the query and the retrieved data. This architecture improves performance in knowledge-intensive tasks like Question Answering (QA), Summarization, and document generation, where the model benefits from having access to external information, rather than relying solely on internal knowledge stored in its parameters.

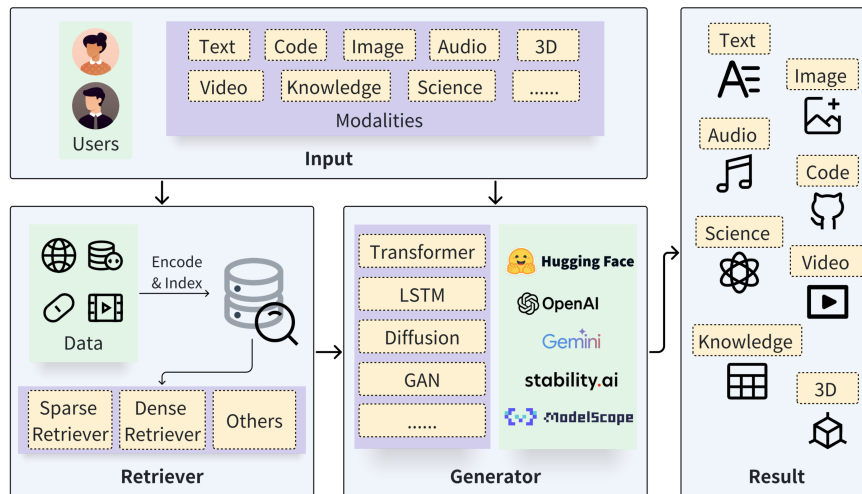


FIGURE 2.1: RAG Architecture Overview (Retrieval-Augmented Generation for AI-Generated Content: A Survey, Zhao et al., 2024 [37]).

Several methodologies for RAG have emerged. Early works like those of Lewis et al. [18] introduced a RAG model that performs document retrieval using dense representations to find related content before generating responses based on retrieved passages. Moreover two approaches were compared, one which uses the retrieved passages for the whole generated sequence and one which uses different passage for each generated token. Generally, the benefits of using RAG architectures are multifaceted. RAG architecture allows LLMs to pull in updated, domain-specific, or specialized knowledge dynamically from external sources, making the system more adaptable. This subsequently prevents these models from having the hallucination problem, which is a significant drawback and may lead to very inaccurate responses and information. Additionally, retrieval-based augmentation reduces the need for retraining or fine-tuning models for new tasks or domains, enhancing efficiency and scalability. Furthermore, the generative component enables handling novel tasks, while the retriever ensures relevant knowledge is integrated into the generative process, improving generalization capabilities.

Fan et al. [9] conducted a thorough survey on various RAG implementations, detailing the different methodologies in terms of prompt learning techniques, retrieval types, Pre and Post retrieval enhancement and benefits of integrating retrieval and generation in large-scale models. However, RAG models also introduce challenges such as integrating retrieval and generation seamlessly and optimizing retrieval mechanisms to improve generation quality. Existing research, including Jeong et al. (2024) [13], highlights the need for identifying the complexity of input queries and then dynamically, adapting the retrieval strategy in accordance to the underlying complexity. Moreover this work aims in handling simpler queries without excessive computational costs while effectively generating appropriate responses for more complex multi-step questions. Also Singhal et al. [28] observed that even though they achieved high accuracy for the averitec dataset on their RAG and ICL based system, they also achieved sub-optimal performance identifying cases where the evidence provided for fact checking were conflicting or not enough.

2.2 Vector Stores: Approaches, Types of Retrievers, and Graph Knowledge Retrieval

Vector Stores are essential in retrieval-based architectures, including RAG, as they hold the vector representations of documents that are later used for retrieval. These stores allow efficient querying by encoding documents into continuous vector spaces. The two primary types of retrieval approaches are dense retrieval and sparse retrieval.

Sparse retrieval methods such as BM25, TF-IDF, and query likelihood models focus on term-based matching, where the system retrieves documents based on exact keyword overlap. These

approaches typically employ inverted indices, which store mappings between terms and documents, enabling efficient lookups based on specific terms or phrases. Sparse retrieval is relatively resource-efficient and works well when query terms closely match the target documents. For example, BM25, as discussed by Zhao et al. (2021) [37], remains a strong baseline in many text retrieval scenarios due to its simplicity and effectiveness in searching documents.

Dense retrieval, encodes documents and queries into vectors in a shared latent space and is a widely-used approach in RAG systems. This allows the system to retrieve semantically similar documents, even when there is no direct lexical overlap between the query and the document. Karpukhin et al. (2020)’s DPR (Dense Passage Retrieval) system [17] is an example of a dense retrieval method that utilizes dual-encoders based on BERT (Bidirectional Encoder Representations from Transformers) to map both queries and documents into a common embedding space. Dense retrieval models allow capturing semantic similarity instead of exact term matching, but they often require large-scale computational resources and may struggle with low-resource settings. However, the strength of dense retrieval lies in its ability to retrieve semantically relevant information beyond simple keyword matching, which is critical for knowledge-intensive tasks in RAG systems.

Extending the previous retrieval methods, hybrid retrieval combines the strengths of both sparse and dense approaches, leveraging term-based exact matching while incorporating semantic similarity through dense representations. These models have gained traction by mitigating the limitations of individual retrieval methods. The hybrid approach effectively balances efficiency and effectiveness by dynamically selecting retrieval strategies. Recent research by Arabzadeh et al.[2] introduces a classifier-based strategy for hybrid retrieval, which optimizes the trade-off between sparse and dense retrieval by dynamically deciding when to incorporate dense representations. This method enhances recall while reducing computational costs by selectively applying dense retrieval only when necessary. Such retrieval methods play a crucial role in retrieval efficiency for knowledge-intensive tasks, particularly in large-scale systems where latency and computational constraints must be carefully managed.

Beyond dense and sparse retrieval, Graph Knowledge Stores represent another retrieval approach that is being explored for use in RAG architectures. In this paradigm, knowledge is stored in graph form, where entities and relations are represented as nodes and edges, respectively. Graph-based retrieval allows the retrieval of semantically rich and interconnected information, making it useful for domains that rely heavily on entity relationships (e.g., biomedical or legal fields). Works like the one of Edge et al. (2024) [7], instead of providing solely text documents to the models, they use LLMs to generate entities and relationships from the provided text documents and by performing a series of community summarizations, the result is provided to the LLM for generating the response. According to the authors this methodology achieved substantial improvements over baseline RAG methodologies. However, graph-based

methods can introduce complexity in scaling and require well-curated graphs for effective retrieval.

One important technique in vector-based retrieval is chunking, which involves splitting long documents or passages into smaller segments (chunks) before indexing. Chunking is particularly useful in dense retrieval systems to ensure the retrieval of granular, relevant information. By splitting content into smaller chunks, the retriever can focus on retrieving specific, contextually relevant passages rather than entire documents, improving the overall retrieval precision. For example, Sentence-window retrieval uses this technique by fetching small chunks of text and returning a window of sentences surrounding the retrieved segment to ensure sufficient context for the generator. This improves the generation's coherence and accuracy by providing a richer context.

Auto-merge retrieval, an advanced technique used in LlamaIndex ¹, organizes documents into a tree-like structure where each node represents a chunk, and parent nodes contain aggregated information from their child nodes. This structure allows the retrieval process to first focus on fine-grained searches within child nodes before returning richer information at the parent node level. This technique effectively balances granularity and context richness.

2.3 Query Prompting Optimization Methodologies

Query prompting in retrieval architectures is critical to optimize the results of the retrieval component in RAG. Several methodologies have been explored in the literature, each offering distinct benefits and drawbacks. An overview of those methodologies is provided below:

- Query expansion
- Query Rewriting
- Prompt Compression
- In-Context Learning (ICL)
- Chain-of-Thought (CoT) Prompting

Query expansion as presented by Wang et al. [30] involves expanding the original query with additional terms or phrases to increase the likelihood of retrieving relevant documents. This can be done either manually or automatically using external knowledge bases. In their work the simple method `query2doc` was used to leverage LLMs for query expansion. They prompt

¹LlamaIndex: An open-source data framework for building applications with large language models, developed by the LlamaIndex community. Available at: https://github.com/run-llama/llama_index

models with few-shot examples in order to generate pseudo-documents and then they integrate them with existing sparse or dense retrievers by augmenting queries with generated pseudo-documents.

Query Rewriting is another technique which, similar to query expansion, optimizes and modifies the user's original query to better match the underlying structure of the data being retrieved from the vector stores. This method has seen widespread use in systems that rely on noisy or under-specified queries. More specifically, Ma et al. [21] introduced the Rewrite-Retrieve-Read framework where they prompt an LLM to optimize the query, and then use a web search engine to retrieve contexts. The authors highlight that this methodology after evaluation performed, was effective on various datasets.

Prompt Compression proposed by Jiang et al. (2023) [14], reduces the length of the query without losing essential information. This method is particularly useful for accelerating model inference, addressing the problem of models getting "lost in the middle" when dealing with long sequences. Compressing the prompt enhances retrieval efficiency by eliminating irrelevant details, thus improving focus on key aspects of the query.

In-Context Learning (ICL) involves providing the model with a few examples of input-output pairs to prompt it to generalize to new, unseen examples without explicit fine-tuning thus providing relevant context. This method has shown significant promise in zero-shot and few-shot learning scenarios, as indicated by Mann et. al. [23] but its performance can be sensitive to the quality and format of the provided context. More specifically, in their work, LLMs were provided with contextual prompts to infer the task at hand and effectively generalize to new queries. The main downside of ICL is its reliance on having high-quality exemplars and its difficulty in managing highly variable tasks or domains.

Chain-of-thought (CoT) prompting is a recent development in query methodologies where the model generates intermediate reasoning steps before producing the final output. CoT allows models to break down complex queries into more manageable parts, improving both retrieval and generation by promoting structured reasoning. However, this technique introduces additional computational overhead and can lead to redundant or overly complex retrieval processes. Wei et al. (2022) [32] highlight that CoT prompting improves performance by a large margin on arithmetic reasoning and that the linguistic nature of chain-of-thought reasoning makes it generally applicable.

Each of these methodologies helps improve retrieval by guiding the model's understanding of the query or context, but they also introduce challenges in terms of computational efficiency and susceptibility to noise.

2.4 Retrieval and LLM Architectures for Generation

In Retrieval-Augmented Generation (RAG) systems, the seamless integration of retrieval and generation is essential for effective knowledge-intensive tasks. The architecture typically comprises a retriever, responsible for identifying relevant passages or documents, and a generator, which synthesizes coherent, contextually rich responses from the retrieved information. The success of these systems hinges on optimizing both the retrieval and generative components, as well as refining evaluation methods to assess their performance. Retrieval in RAG models begins by identifying and retrieving relevant documents or passages from external corpora or knowledge bases. Several advancements in retrieval methodologies have emerged to enhance the precision and relevance of the retrieved information:

- HyDE (Hypothetical Document Embeddings) - Gao et al. [11]
- RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) - Sarthi et al. [27]
- SelfRAG - Asai et al. [3]
- LongRAG - Jiang et al. [15]
- FLARE (Forward Looking Active Retrieval) - Jiang et al. [16]
- GraphRAG (Graph-based RAG for Query-Focused Summarization) - Edge et al. [7]
- Persona-Based Neural Conversation Model - Li et al. [19]

HyDE (Hypothetical Document Embeddings), proposed by Gao et al. [11], focuses on generating pseudo-documents from the given queries, which are then used to retrieve relevant passages. By synthesizing richer content from the initial query, HyDE captures more nuanced or semantically relevant documents, improving retrieval effectiveness in both dense and sparse retrieval settings. This approach ensures that the retrieval process is more robust to ambiguities in the query, ultimately leading to better downstream generation.

Sarthi et al. [27] designed the RAPTOR (Recursive Abstractive Processing for Tree-Organized Retrieval) which operates using a tree-structured retrieval approach, where documents are hierarchically organized into chunks at varying levels of granularity. At each level of the hierarchy, summarization is performed on the clustered documents using GPT-3.5-turbo, with the parent node containing the summarized information from the child nodes. This recursive process continues, performing clustering and summarization, until further clustering becomes infeasible, ensuring the system can scale linearly with the data. During retrieval, RAPTOR uses tree traversal. The system first retrieves top-k documents by calculating cosine similarity

between the query embedding and the nodes at the root level and repeats this process across the tree structure.

SelfRAG introduced by Asai et al. [3] add a self-reflective mechanism to the standard RAG process, where the model evaluates and critiques its own retrieved passages before using them for generation. By doing so, SelfRAG ensures that only the most relevant information is passed to the generator, reducing hallucinations and increasing the factual accuracy of the generated responses.

LongRAG proposed by, Jiang et al. [15] is a framework for tasks requiring long-context understanding, that enhances traditional retrieval methods by incorporating longer contextual windows in both retrieval and generation. This allows LongRAG to handle multi-turn conversations or complex queries that require information spread across multiple documents or longer passages, addressing a limitation in standard retrieval methods where long-distance dependencies are overlooked.

Moreover Jiang et al. [16] designed FLARE (Forward Looking Active Retrieval) which introduces a novel approach to retrieval-augmented generation by iteratively retrieving relevant information during the generation process, rather than relying on a single retrieval step before generating the output. Unlike traditional RAG models, which retrieve information once based on the input query, FLARE anticipates future content needs by generating a prediction of the upcoming sentence and using it as a query to retrieve additional relevant documents. This forward-looking approach allows FLARE to dynamically adjust its retrieval strategy as the generation unfolds, ensuring that each generated sentence is informed by up-to-date and contextually relevant information. As highlighted by the authors FLARE enhances both the precision and depth of responses, making it well-suited for complex reasoning and explanation-driven tasks.

GraphRAG, proposed by Edge et al. [7], introduces a novel approach for global query-focused summarization in RAG systems by leveraging a graph-based text index. Unlike traditional RAG approaches that retrieve local passages directly from external text, GraphRAG utilizes an LLM-derived knowledge graph that is constructed in two stages: first by deriving an entity knowledge graph from the source documents and then by pre-generating community summaries for groups of closely-related entities.

For query answering, these community summaries generate partial responses, which are subsequently combined into a final answer. This approach significantly improves the comprehensiveness and diversity of the generated answers for global questions that make sense, over large datasets, addressing inherent limitations of traditional RAG approaches that struggle with corpus-level summarization.

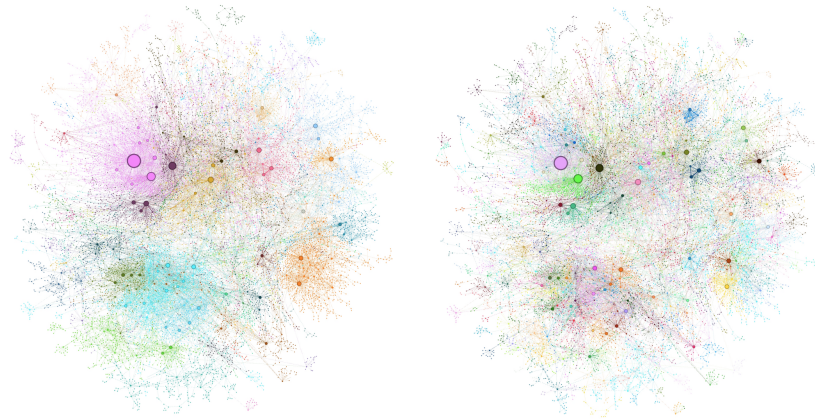


FIGURE 2.2: Communities in GraphRAG during summarization step (From Local to Global: A Graph RAG Approach to Query-Focused Summarization, Edge et al., 2024). [7]

In terms of improving the generation process, a Persona-Based Neural Conversation Model by Li et al. (2016) [19] addresses the challenge of maintaining speaker consistency in neural conversation models. The authors propose persona-based models that incorporate distributed embeddings to encode individual characteristics such as background information and speaking style. This ensures that the generated responses are not only coherent but also consistent with the speaker’s persona. Additionally, the paper introduces a dyadic speaker-addressee model, which captures interaction dynamics between two interlocutors, further enhancing the quality of conversations. Their approach demonstrates significant improvements over standard sequence-to-sequence models in terms of perplexity, BLEU scores, and speaker consistency. This work has laid the groundwork for incorporating personalized and context-aware dialogue generation in neural systems, making it highly relevant for conversational agents.

2.5 Evaluation Methods

Evaluating the effectiveness of a Retrieval-Augmented Generation (RAG) system requires a comprehensive approach that assesses both retrieval accuracy and the quality of generated responses. Traditional evaluation metrics, such as BLEU and ROUGE for text generation and Precision@k or Recall@k for retrieval, fail to capture the interplay between retrieval and generative output, particularly in knowledge-intensive tasks.

Yang et al. (2024) [35] introduce the CRAG benchmark, which addresses these limitations by incorporating a scoring system that penalizes hallucinations, distinguishes missing answers, and evaluates factual consistency. Their approach highlights the importance of assessing retrieval and generation jointly rather than in isolation. Furthermore, reducing hallucination remains a key challenge in the evaluation of RAG systems. Moreover, CRAG provides a rich evaluation framework with 4,409 question-answer pairs, covering five diverse domains—Finance,

Sports, Music, Movies, and an Open domain—across eight different question categories. The benchmark highlights the diverse and dynamic challenges faced by RAG systems, including entity popularity (ranging from head to tail entities) and temporal dynamism (from real-time to static facts). Evaluation using CRAG demonstrated that even advanced RAG systems achieve only moderate gains in accuracy—from 34% with LLM-only solutions to 44% when using RAG—indicating the need for more effective retrieval and synthesis approaches.

Recent innovations like SelfRAG by Asai et al. [3] introduce the SELF-RAG framework, which integrates a self-evaluation mechanism where the model dynamically re-evaluates retrieved passages during generation. Unlike conventional RAG approaches that retrieve a fixed number of passages without assessing their relevance, SELF-RAG adaptively determines when retrieval is necessary and critiques both the retrieved content and its own generated outputs. Through reflection tokens—such as retrieval (Retrieve), relevance assessment (ISREL), and factual support verification (ISSUP)—the model iteratively refines its responses, thereby reducing hallucinations and enhancing factual accuracy. Empirical evaluations demonstrate that SELF-RAG significantly outperforms state-of-the-art retrieval-augmented LLMs, improving citation accuracy and factual consistency in open-domain QA, reasoning, and fact verification tasks.

Yu et al. (2024) [36] emphasized the challenge of evaluating RAG systems due to their hybrid architecture, which relies on both retrieval and generative capabilities. To address this, they proposed A Unified Evaluation Process of RAG (Auepora), a comprehensive evaluation framework that systematically analyzes retrieval and generation components using metrics such as relevance, accuracy, and faithfulness. Auepora underscores the importance of capturing both retrieval efficiency and generative quality in a unified benchmark, thereby providing a clearer understanding of system performance and identifying areas where current methodologies fall short.

Finally, Es et al. [8] introduce RAGAS (Retrieval-Augmented Generation Assessment), an automated evaluation framework designed for reference-free assessment of Retrieval-Augmented Generation (RAG) systems. Unlike conventional RAG evaluation methods that rely on ground-truth annotations, RAGAS employs a suite of self-contained metrics to assess retrieval quality and generative accuracy. The framework evaluates three core dimensions: faithfulness (ensuring generated responses align with retrieved evidence), answer relevance (measuring how well responses address the query), and context relevance (ensuring retrieved passages contain minimal irrelevant information). RAGAS integrates seamlessly with existing RAG development frameworks such as LlamaIndex and LangChain, enabling rapid evaluation cycles and improving the reliability of RAG pipelines in real-world applications. This framework is extensively used in this thesis with more details provided on Chapter 4.

2.6 LLM based Application on the Tourism Domain

Large Language Models (LLMs) have found significant applications in the tourism domain, where personalization, information retrieval, and user interaction are critical. Various works explore how LLMs can enhance user experiences by providing accurate, context-aware, and personalized responses. In the LLM-based Tourism Applications domain, one prominent work, TourLLM by Wei et al. [33], focuses on enhancing LLMs with tourism knowledge, creating a specialized dataset comprising knowledge-based QA, travelogues, and diverse QA data. The authors used prompt-based templates to generate question-answer pairs, manually refining and employing parameter fine-tuning on a specific LLM to avoid hallucination. Evaluation metrics included consistency, readability, and availability, all relevant to Chinese-language tourism applications, though this work notably did not employ a RAG framework implementation. This highlights the utility of domain-specific knowledge and datasets in overcoming LLM limitations in tourism contexts.

Additionally, VirtuWander by Wang et al. [31] utilized LLMs in a multi-modal virtual tour guidance setting for art museums. Their model framework classified the context (e.g., start, progress, end) and tailored responses based on task and information type (spatial, semantic, social). This multi-agent system enhances personalized interactions without using explicit RAG structures, instead leveraging few-shot learning and GPT-based paradigms to dynamically generate feedback.

In contrast, RAG-Optimized Tibetan Tourism LLMs by Qi et al. [25] demonstrates the direct application of RAG in tourism. Sparse and dense retrieval methods, including TF-IDF and BERT-based vectorization, are used to enhance accuracy and personalization in Chinese-language tourism contexts. Their system integrates a hierarchical navigable small-world network (HNSW) for efficient retrieval, improving both the fluency and relevance of the generated responses. Additionally, Qi et al. [24] introduce the DualGen Bridge AI system, a large language model (LLM)-based framework designed to enhance the accuracy and personalization of Tibetan tourism viewpoint information. The system employs a multi-phase approach, beginning with location keyword extraction using fine-tuned LLMs, followed by a bridge model that maps user queries to geographically relevant tourist spots. The final phase involves an LLM tourist viewpoint generation model that synthesizes structured content about destinations. The framework incorporates Supervised Fine-Tuning (SFT), Low-Rank Adaptation (LoRA), and Odds Ratio Preference Optimization (ORPO) to optimize model performance. Empirical evaluations validate the effectiveness of these techniques, demonstrating improvements in fluency, accuracy, and relevance of generated content. This approach highlights the potential of fine-tuned LLMs in generating high-quality tourism content tailored to user preferences in a geographically diverse region.

Other approaches in terms of Context-Aware and Graph-Based Models, like CHAT-Bot by Casillo et al. [4], emphasize the integration of context-aware frameworks using models for personalized touristic experiences. Their chatbot utilizes a Context Dimension Tree for querying relevant data and personalizing responses. Although this architecture does not rely on RAG, it demonstrates the value of such models and contextual analysis for refining user interactions, particularly in long and complex dialogue sessions.

Machidon et al. [22] introduced CulturalERICA, a conversational agent designed to enhance the exploration of European cultural heritage through natural language interaction. The proposed system integrated Dialogflow-based AI, a custom Web Service, and Google Assistant to provide an accessible interface for querying Europeana's vast digital heritage database. CulturalERICA employed a ranking algorithm combining statistical metadata analysis and computer vision techniques to improve search result relevance. Empirical evaluation demonstrates that the ranking mechanism increases the proportion of relevant search results and enhances user satisfaction, as confirmed by a user study employing Technology Acceptance Model (TAM) and Software Usability Scale (SUS) methodologies. The study underscores the potential of intelligent conversational agents in improving access to digital cultural heritage repositories.

In terms of persona use cases, Liu et al. [20] introduce a content-oriented user modeling approach for personalized response ranking in chatbots. Their method leverages a two-branch neural network to learn latent user representations from user-generated content (UGC), incorporating several implicit characteristics. Afterwards, these personalized embeddings are integrated into a deep neural network framework for response ranking, optimizing chatbot interactions from the user's perspective.

Furthermore, Li et al. [19] introduce a persona-based neural conversation model that enhances speaker consistency in response generation. The proposed framework incorporates two models: the Speaker Model, which encodes individual speaker characteristics in distributed embeddings, and the Speaker-Addressee Model, which captures interaction patterns between interlocutors. These models are integrated within a sequence-to-sequence (SEQ2SEQ) framework, enabling more personalized and coherent responses. According to the authors, experimental results demonstrate that leveraging persona embeddings improves BLEU scores by up to 20% and perplexity by 12% compared to baseline models. Overall, these works underscore the potential of LLMs in tourism, where retrieval mechanisms, context-aware processing, and fine-tuning contribute to enhanced user experiences. However, the integration of RAG architectures in tourism remains an emerging research direction, with further studies needed to explore its full potential.

Overall, in this chapter, related past literature was provided in terms of RAG relevant topics and from the aspect of building such architectures in domain-specific use cases and evaluating them.

Title	IA	MRM	LG	RAG	PI
TourLLM: Enhancing LLMs with Tourism Knowledge			✓		
VirtuWander: Enhancing Multi-Modal Interaction for Virtual Tour Guidance Through Large Language Models	✓		✓		
RAG-Optimized Tibetan Tourism LLMs: Enhancing Accuracy and Personalization		✓	✓	✓	
Research on Tibetan Tourism Viewpoints Information Generation System Based on LLM			✓	✓	
CHAT-Bot: A Cultural Heritage Aware Teller-Bot for Supporting Touristic Experiences	✓				
CulturalERICA: A Conversational Agent Improving the Exploration of European Cultural Heritage					
Content-Oriented User Modeling for Personalized Response Ranking in Chatbots		✓			
A Persona-Based Neural Conversation Model	✓		✓		✓
Enhancing Personalized Travel Assistance with Retrieval-Augmented Generation: A Context-Aware Approach - TouaRAG	✓	✓	✓	✓	✓

TABLE 2.1: Categorization of domain-specific papers based on their contributions to Information Augmentation (IA), Multiple Retrieval Methodologies (MRM), LLM Generation (LG), RAG, and Persona Integration (PI).

The insights gained from the cited works and as presented in Table 2.1 was essential to proceed with the creation of the proposed RAG framework implemented in this thesis which covers all the aforementioned needs for Information Augmentation, Multiple Retrieval Methodologies experimentation, LLM Generation incorporation, RAG Architectures implementation and Persona Integration within the procedures of the pipeline. All those aspects, are discussed thoroughly in Chapters 4 and 5 which aim to enhance the overall tourism experience, as discussed in Chapter 1. In the next Chapter we present in more detail the main principles and fundamentals related to RAG architectures.

Chapter 3

RAG Fundamentals

In this chapter, definitions of terms used throughout the thesis are introduced. More definitions are introduced in other chapters (where needed). In addition, the basic concepts of the project architecture are presented.

3.1 Principles and Fundamentals

In this section, several fundamental term definitions related to RAG architecture implementations, which are used throughout this thesis, are introduced.

3.1.1 Retrieval

RAG frameworks rely on retrieval processes to obtain relevant information from external knowledge bases, which can be either open-source or proprietary. The most critical component of this process is the retriever, which incorporates several techniques for fetching relevant information from these knowledge sources. Retrievers aim to retrieve the top- k most similar results for a given user query q using a similarity function s . The specific pre- and post-retrieval techniques used can enhance the retrieval pipeline and may vary based on the application.

The two main types of retrievers, sparse and dense, are discussed in more detail in this section. Sparse retrieval, which is used in the experiments described in this thesis, is explored in the following subsection.

3.1.1.1 Sparse Retrieval

Sparse retrieval is primarily applied to text retrieval and focuses on encoding text into sparse vectors. This process often relies on the vocabulary length of tokenized words. Sparse retrieval techniques typically utilize inverted indices for efficient lookup and emphasize the importance of terms based on their occurrences. Commonly used sparse retrievers include TF-IDF [29] and BM25 [26], both of which represent encoded passages as a bag of words and rank results based on inverted indices. BM25, in particular, computes a score for each document-query pair based on term frequency, document length normalization, and inverse document frequency. It strikes a balance between these factors to provide a computationally efficient and effective ranking mechanism. BM25 is widely used in search engines, question-answering systems, and other information retrieval tasks. In the context of RAG systems, BM25 ensures that the retriever provides highly relevant passages to the generator.

The BM25 relevance score for a document D with respect to a query Q is computed as:

$$\text{Score}(D, Q) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgD}}\right)}, \quad (3.1)$$

where t is a term in the query Q , $f(t, D)$ is the term frequency of t in the document D , $|D|$ is the length of the document D (e.g., number of terms), avgD is the average document length in the corpus, k_1 and b are hyperparameters controlling term frequency saturation and length normalization, respectively, and $\text{IDF}(t)$ is the inverse document frequency of term t .

The inverse document frequency is computed as:

$$\text{IDF}(t) = \log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right), \quad (3.2)$$

where N is the total number of documents in the corpus, and n_t is the number of documents containing the term t . The IDF [29] component assigns higher weights to terms that appear in fewer documents, thereby emphasizing rare but potentially significant terms. The term frequency $f(t, D)$ is adjusted using the k_1 parameter to account for diminishing returns in relevance as term occurrences increase. The length normalization component, controlled by the b parameter, ensures that longer documents are not unfairly penalized or favored. Specifically, k_1 determines the extent of term frequency saturation, where higher values of k_1 reduce the impact of term frequency on the score. The parameter b controls the influence of document length normalization, with $b = 1$ implying full normalization and $b = 0$ disabling it. To illustrate, sparse retrieval techniques like BM25 serve as foundational tools for ensuring that relevant and concise passages are retrieved, enabling downstream components of RAG systems to generate coherent and contextually accurate outputs.

3.1.1.2 Dense Retrieval

Dense retrieval represents both queries and documents as dense embedding vectors in a continuous vector space, enabling the capture of rich semantic relationships between textual inputs. Unlike sparse retrieval, which relies on discrete term matching, dense retrieval utilizes deep learning models to encode and retrieve data based on semantic similarity. This paradigm extends across multiple modalities, including text, code, audio, images, and video, making it a versatile approach for retrieval-augmented generation (RAG) systems.

Dense retrieval systems use pre-trained or fine-tuned neural network architectures to generate high-quality embeddings. Transformer-based models such as BERT and RoBERTa are commonly employed for encoding queries and documents. Bi-encoder architectures, with separate encoders for queries and documents, are particularly popular due to their efficiency and scalability. Furthermore, specialized models have been developed to encode non-textual modalities, such as code, audio, images, and video, enabling dense retrieval to support diverse data sources in RAG architectures. As a model in dense retrieval, Dense Passage Retriever (DPR) [17] uses a BERT-based backbone and is pre-trained on question-answer pairs for open-domain question answering. It excels in aligning query and document embeddings, enabling precise and task-specific retrieval. Dense retrieval systems are typically trained using contrastive learning, which maximizes the similarity between positive query-document pairs while minimizing the similarity for negative pairs. Dense retrieval also faces challenges in scalability, particularly during inference. To address this, approximate nearest neighbor (ANN) methods are employed, leveraging specialized indices for efficient searching. Moreover, tree-based indices use techniques such as KD-trees to provide hierarchical structures for rapid search. Another option, graph-based indices, employ methods like HNSW, DiskANN, and HMANN, which use proximity graphs to organize embeddings for scalable approximate nearest neighbor search. Finally, hybrid indices, such as SPANN, combine graph and inverted index structures to optimize retrieval efficiency.

Dense retrieval is a foundational component of RAG systems, enabling the retrieval of semantically relevant information for downstream generation tasks. Dense retrievers are particularly effective in scenarios requiring semantic understanding, such as open-domain question answering, conversational AI, and recommendation systems. In RAG frameworks, dense retrieval in some cases complements sparse retrieval, combining semantic richness with exact match efficiency for enhanced performance. This approach is also experimented in this thesis and explained in Section 5.2.2. Despite its significant advantages, dense retrieval can be computationally intensive due to the high dimensionality of dense embeddings and the need for extensive pre-training. Recent advancements aim to address these challenges with hardware acceleration and distributed computing techniques which are improving the scalability of

dense retrieval systems. Additionally research into lightweight embedding models and quantization methods offers potential for reducing computational overhead. By leveraging advanced architectures, robust training techniques, and efficient indexing methods, dense retrieval remains a pivotal component in modern RAG systems, offering flexibility and semantic depth.

3.1.2 Pre-Post Processing Enhancement

Retrieval-Augmented Generation (RAG) architectures rely heavily on the interplay between retrieval mechanisms and generation processes. Ensuring the alignment and quality of information at various stages is critical for system performance. Enhancements in pre-retrieval (query-level) and post-retrieval (retrieved information processing) stages aim to optimize the accuracy, relevance, and usability of information fed into and extracted from RAG systems. The incorporation of pre- and post-processing strategies addresses several critical challenges leading to benefits in RAG systems. The first is improved relevance, where enhanced queries and refined retrieval results lead to higher-quality document selection. Another one is reduced noise which is achieved through filtering mechanisms that mitigate the impact of irrelevant or misleading data. Moreover, generator alignment ensures better compatibility between retrieved content and generator requirements through tailored post-retrieval processing. Finally, system efficiency is enhanced by compression and summarization techniques that reduce computational overhead, enabling more efficient generation. By adopting these pre- and post-processing techniques, RAG systems can achieve enhanced retrieval accuracy and generation quality, making them better equipped to handle complex, real-world tasks. These enhancements can significantly boost overall effectiveness by addressing challenges such as query ambiguity, irrelevant data, and noise in retrieved content. In the following sections, we explore these enhancements in detail.

3.1.2.1 Pre-Retrieval Enhancement

Pre-retrieval enhancements focus on improving the initial query presented to the retrieval module. By refining or augmenting the user's input, these techniques aim to maximize the relevance of retrieved documents.

- Pseudo-Document Generation
- Query Rewrite
- Query Augmentation
- Noise Mitigation

- Data Augmentation

Pseudo-document generation methods, such as Query2doc [30] and HyDE [11], enhance queries by generating hypothetical documents that encapsulate richer, contextually more related information to the original user query. These documents act as enhanced queries, significantly improving the retrieval performance for both sparse and dense retrievers. Moreover, Query Rewrite frameworks, such as Rewrite-Retrieve-Read, reformulate ambiguous or underspecified queries into more precise versions [21]. This reformulation bridges gaps between user intent and retriever comprehension, resulting in improved alignment and retrieval quality. Another method is, Query Augmentation which combines the original query with auxiliary information, such as preliminary outputs or related context. This way, the augmented query better aligns with the retriever's requirements, thereby retrieving documents with higher lexical and semantic relevance. Noise mitigation techniques in pre-retrieval strategies often involve filtering irrelevant or outdated content before querying. Llamaindex¹ package which is extensively used in this thesis, includes in its implementation various functionalities for retrieving only relevant chunks, ensuring any irrelevant or "noisy" chunks are filtered out. Finally, Data Augmentation in preprocessing steps, such as synthesizing new data, updating outdated documents, or enriching original content with various metadata, enhances the initial data quality. In this thesis, Data Augmentation through metadata enrichment is used and explained for both the data generation methodology and the RAG querying methodology in detail in Chapter 5.

3.1.2.2 Post-Retrieval Enhancement

Post-retrieval enhancements process the retrieved content before feeding it into the generator. These techniques are critical for aligning retrieved knowledge with the generation stage and addressing issues such as noise, redundancy, or length limitations.

- Re-Ranking Retrieved Results
- Compression and Summarization
- Filtering and Adaptation
- Chunk Optimization

The Re-ranking retrieved results category of post-retrieval enhancement involves methods like Retrieve-Rerank-Generate (R2G), which reorder retrieved documents to prioritize those with higher relevance or diversity. This reordering ensures robustness in retrieval outcomes and

¹LlamaIndex: An open-source data framework for building applications with large language models, developed by the LlamaIndex community. Available at: https://github.com/run-llama/llama_index

reduces the likelihood of noisy or irrelevant data adversely affecting generation. Compression and summarization techniques like the one presented by Jiang et al. [14], such as Retrieve, Compress, Prepend (RECOMP), summarize retrieved content to fit within the token constraints of large language models. This intermediate step enables the efficient use of lengthy documents while preserving key information for generation. Furthermore, Filtering and adaptation techniques employ metadata-based filtering, utilizing attributes such as timestamps or source types to exclude irrelevant documents. For instance, if a user query pertains to a specific subset of available documents, targeted filtering to those documents can be applied to ensure that only the most relevant information is fed into the system. This enhances retrieval efficiency by reducing noise and focusing on more appropriate content. Finally, Chunk optimization improves context and relevance by splitting and optimizing document chunks during retrieval. Sentence-window retrieval, for example, retrieves smaller text fragments while preserving surrounding context, resulting in a more comprehensive understanding.

3.1.3 Evaluating RAG Architectures

Evaluating Retrieval-Augmented Generation (RAG) architectures is critical for understanding their performance, efficiency, and suitability for specific applications. Evaluation involves assessing both retrieval and generation components, as well as their integration within the broader system. Metrics, datasets, and methodologies tailored to different use cases are essential for comprehensive analysis.

- Retrieval Metrics
- Generation Metrics
- End-to-End Metrics
- LLM Based Metrics

Initially, retrieval metrics such as $\text{precision}@k$, $\text{recall}@k$, and mean reciprocal rank (MRR), as also discussed in Chapter 7 assess the quality of retrieved documents in meeting query requirements. Afterwards, generation metrics, including BLEU, ROUGE, and METEOR, measure the fluency and relevance of generated outputs, while metrics like BERTScore and faithfulness scores evaluate semantic alignment and factual accuracy. More details on the specific metrics used in this thesis can be found in Section 6.3. Another option of metrics is the End-to-end ones, such as human evaluations, user satisfaction scores, and task-specific success rates, which provide holistic assessments of system performance. The last methodology, instead of human evaluations, it employs a reference-free approach, as seen in the works of Es et al. [8], where

LLM-as-judge is used to perform various metric extractions without explicitly annotated text. This methodology is extensively used in this thesis and is further explained in Section 6.3.

Overall, this chapter introduced the notation, fundamental principles, and core components of RAG architectures. Key concepts such as retrieval mechanisms, including sparse and dense retrieval, as well as pre- and post-processing enhancements, were explored in detail. Additionally, evaluation methodologies for assessing RAG systems were discussed, highlighting various retrieval, generation, and end-to-end metrics. The next chapter will build upon this theoretical foundation by presenting the practical implementation of these concepts, offering a comprehensive description of the platform's architecture and its integration with the methodologies discussed herein.

Chapter 4

TouaRAG Tools & System Architecture

As mentioned in Chapter 1, the project of this thesis aims to develop a RAG system which will enhance user experience and make trip planning a seamless and enjoyable process. To accomplish this, knowledge and techniques were incorporated from the fields of Artificial Intelligence (AI), Information Retrieval (IR) and Computational Linguistics (CL). In this chapter the system's general architecture along with presentation of tourism specific dataset created will be presented.

4.1 Development Libraries and Frameworks

The implementation of Retrieval Augmented Generation (RAG) system for enhancing tourism experiences relied on a diverse set of libraries and frameworks. These tools provided the necessary infrastructure for handling natural language processing, information retrieval, knowledge graph generation, and system evaluation. Below, we detail the key libraries and their roles in the project.

4.1.1 Llamaindex

LlamaIndex ¹ played a central role in building and managing the retrieval components of the RAG system. It provided essential tools for document parsing, enabling the efficient processing of large datasets such as travel guides and brochures into structured nodes. Additionally, it supported entity extraction by identifying key entities and their relationships, which contributed

¹LlamaIndex: An open-source data framework for building applications with large language models, developed by the LlamaIndex community. Available at: https://github.com/run-llama/llama_index

to the construction of the knowledge graph. To facilitate retrieval, LlamaIndex incorporated vector indexing wrappers, allowing document chunks to be transformed into vector representations and efficiently stored for retrieval.

4.1.2 Hugging Face API

Hugging Face [34] was integral to leveraging several open-source large language models (LLMs) available through its platform. It provided access to pre-trained transformer models, which were used for language understanding and embedding generation. Additionally, it facilitated custom fine-tuning by allowing the adjustment of hyperparameters and the application of quantization techniques to reduce memory and computational demands, ensuring alignment with implementation constraints. Furthermore, Hugging Face's tokenizer support enabled efficient text preprocessing, ensuring compatibility with different models and optimizing input representation.

4.1.3 OpenAI API

OpenAI's API ² provided access to advanced language models, enabling both natural language understanding and generation through transformer-based architectures. These models helped the comprehension of textual input and the generation of coherent responses, making them fundamental to various retrieval-augmented generation tasks. Additionally, the API was used for embedding creation, allowing the production of dense vector representations for text. These embeddings played a crucial role in the experimental results shown in Section 6.4.

4.1.4 PostgreSQL with PGVector

PostgreSQL ³, extended with the PGVector module ⁴, was used as the primary vector store for most architectural scenarios in this thesis. PGVector offers several advantages that make it well-suited for retrieval-augmented generation tasks. As a native PostgreSQL extension, it allows seamless integration with the database system, enabling the use of PostgreSQL's robust features such as transactions, access control, backup, and indexing alongside vector storage. It also supports efficient vector operations, including similarity search methods such as cosine similarity and nearest neighbor search, reducing the complexity of managing multiple

²OpenAI API: A comprehensive interface for accessing OpenAI's language models, enabling tasks such as text generation, translation, and summarization. Available at: <https://platform.openai.com/docs/api-reference/introduction>

³PostgreSQL: A powerful, open-source object-relational database system developed by the PostgreSQL Global Development Group. Available at: <https://www.postgresql.org/>

⁴pgvector: An open-source PostgreSQL extension for vector similarity search, developed by the pgvector community. Available at: <https://github.com/pgvector/pgvector>

databases in conventional applications. Additionally, PGVector provides specialized indexing mechanisms that enhance query performance and scalability, ensuring efficient handling of large-scale vector data.

4.1.5 RAGAS

The RAGAS library [8] provided essential tools for evaluating and optimizing the proposed RAG system by enabling objective, data-driven assessments. It contributed to synthetic dataset generation by creating evaluation datasets with user personas and queries, ensuring that test scenarios closely aligned with real-world applications. Additionally, RAGAs offered a robust set of evaluation metrics to measure system performance across multiple dimensions, including factual correctness, semantic similarity, and relevance. Beyond these core functionalities, RAGAS served as a comprehensive toolkit for evaluating large language model applications. It provided both LLM-based and traditional evaluation metrics, enabling precise assessments of model behavior. The library also supported automated test data generation, producing diverse and production-aligned datasets to enhance model robustness. Seamless integrations with popular LLM frameworks, such as Llamaindex, and major observability tools further extended its applicability. These capabilities made RAGAS a crucial component for the systematic evaluation and optimization of the RAG architectures explored in this thesis.

4.1.6 GraphRAG

Microsoft’s GraphRAG library [7] played a key role in incorporating knowledge graph-based retrieval within the proposed RAG system. This ready-to-use implementation automated the knowledge graph creation process by processing input data and leveraging a language model to extract nodes and relationships from the provided text. It also facilitated the construction of community hierarchies by clustering nodes based on semantic similarity, ensuring a structured representation of information. Additionally, GraphRAG integrated an automated process for identifying the most relevant communities based on user queries and performing community summarization. More details about the GraphRAG approach in this thesis will be presented in Section 5.2.5.

4.2 General Architecture of System

Aligning with the architecture of a RAG system, the proposed system is firstly based on the query provided by the user. This query is fed to the retrieval procedure, by creating vector representations of the aforementioned query and is using it for identifying relevant information.

More fundamental information on the retrieval procedure is provided in previous chapter. Afterwards relevant information along with the user query is streamlined to the corresponding LLM and then an answer is provided. In general, the system employs a sophisticated combination of Information Retrieval (IR) and LLMs to deliver enhanced tourist guide recommendations.

- Document Corpora
- Knowledge Base
- Retrieval Module
- Response Synthesis

More specifically, the system utilizes a diverse document corpus consisting of tourist-related documents, either provided by users or fetched from external sources. These documents are parsed into formats compatible with the system and stored in a database that serves as the primary knowledge source. The documents may include tourist guides, historical information about specific locations, reviews of tourist attractions, and other tourism-related content. Additionally, the system identifies and stores metadata based on the original source of the documents and extracted entities. Then, the extracted information is organized within a structured knowledge base, which can take the form of relational databases, document stores, vector databases, or knowledge graphs. This way, the knowledge base remains up-to-date by continuously integrating new sources and information, ensuring its relevance over time. This thesis employs all these types of knowledge bases in different RAG approaches. Afterwards, the retrieval module is responsible for fetching relevant content based on user queries. It employs sparse retrievers, such as BM25, for token-based matching, as discussed in Chapter 3 Section 3.1.1.1, and dense retrievers for semantic matching in the embedding space. To further enhance retrieval performance, hybrid strategies, including reciprocal rank fusion and relative scoring, are utilized to combine both approaches. When a knowledge graph-based approach is applied, the system retrieves related entities and relationships rather than solely fetching directly relevant content. This enables contextual expansion of retrieved information by leveraging graph properties such as neighborhood relations, entity types, and community detection. Generally, the ranking of retrieved contexts across all retrieval methodologies is performed based on various metrics, including cosine similarity in embedding space, graph-based community detection, and TF-IDF scores for sparse retrieval. Furthermore, once relevant contexts are retrieved and ranked, response synthesis is performed using a pre-trained large language model (LLM). This LLM incorporates the retrieved information as input to generate responses that are both factual and contextually grounded. RAG systems often integrate domain-specific fine-tuning or prompt engineering to optimize the quality of responses for specific applications. In this

thesis, prompts were carefully engineered to ensure that the generated responses align with the requirements of the tourism domain. Finally, the system undergoes evaluation to assess the accuracy and robustness of both the retrieved contexts and the generated answers. Various evaluation metrics are employed depending on the specific component being analyzed within the information retrieval and response generation pipeline. This ensures that the system maintains high performance and produces reliable responses.

An indicative workflow that applies for all approaches implemented in this thesis is presented in summary, below:

1. Data Preparation
2. Query Processing
3. Context Retrieval
4. Response Generation
5. Response Evaluation

The system follows a structured pipeline, beginning with data preparation, where source documents are ingested, preprocessed, and indexed using dense and/or sparse retrieval methods. If GraphRAG is utilized, knowledge graph structures are incorporated into the indexing process to enhance retrieval efficiency. Once the data is prepared, query processing takes place, where user queries are tokenized, enriched, and matched against the indexed knowledge base or graph to ensure optimal retrieval of relevant information. Following query processing, context retrieval is performed using various strategies, including sparse, dense, hybrid, or graph-based retrieval methods, with GraphRAG enabling structured knowledge graph-based retrieval. The retrieved contexts are then passed to a generative language model during the response generation phase, where the model synthesizes the final output based on the retrieved information. Finally, response evaluation is conducted, either manually or through automated techniques, to assess the quality and accuracy of generated responses. This evaluation phase helps refine both the retrieval and generation models, ensuring improved system performance and alignment with user expectations.

4.3 Synthetic Test Data Generation

In order to evaluate the RAG system presented in this thesis, a Knowledge Graph based methodology was incorporated through the usage of RAGAS library. This library supports handling various and diverse query types during data generation in RAG systems which will be explained below.

- Single-Hop Queries
- Multi-Hop Queries

Queries in retrieval-augmented generation systems can be categorized based on their complexity and the number of information sources required for answering them. Single-hop queries involve straightforward questions that retrieve information from a single source. These queries can be either specific, such as "What year did Albert Einstein publish the theory of relativity?", or abstract, such as "How did Einstein's theory change our understanding of time and space?". In contrast, multi-hop queries require reasoning across multiple sources, often involving multiple nodes connected through relationships in a knowledge graph. Like single-hop queries, they can also be specific, such as "Which scientist influenced Einstein's work on relativity, and what theory did they propose?", or abstract, such as "How have scientific theories on relativity evolved since Einstein's publication?". Multi-hop queries introduce additional complexity by necessitating information synthesis from different sources, making them particularly well-suited for knowledge graph-based retrieval approaches.

For enriching the knowledge graph, built-in functions of RAGAS library exist, that perform various transformations including summary extraction, keyphrases extraction, titles and headlines extraction, and embeddings, as well as building similarity relationships between nodes. Which of those transformations are used is algorithmically set in RAGAS library and is dependent to the token number of each document. Moreover the RAGAS library provides the ability to generate synthetic data by defining a persona with the name and role description of the different persona's that might be relevant to each use case. This is enabled through prompt engineering techniques that were incorporated for their persona module. Finally, the RAGAS library incorporates the scenario generation module. It consists of parameters like the nodes which are used to generate the graph, the desired query length, the query style (e.g. chat, web search etc.) and the personas defined. The process begins by identifying relevant nodes in the knowledge graph. For example, pairs of nodes that share a common property, like an entity, are selected as the basis for generating scenarios. These pairs are then used to create a variety of combinations by varying other parameters, such as query length and style, to ensure diverse coverage. Each scenario is further expanded to include a query, a relevant context from the knowledge graph, and an appropriate reference answer. This ensures that the resulting test data represent a wide range of query styles and complexities, supporting thorough evaluation of the RAG system across various use cases. In general, the synthetic test data creation workflow consists of the following steps:

- Knowledge Graph Creation
- Knowledge Graph Enrichment

- Scenario and Query Generation

The process begins with knowledge graph creation, where documents are split into nodes and the initial graph structure is established. Once the basic structure is in place, knowledge graph enrichment is performed by adding semantic details to the nodes, constructing relationships between entities, and applying transformations to enhance the overall representation of information. Following this, scenario and query generation is carried out by synthesizing realistic scenarios, generating diverse queries, and saving test datasets. This structured pipeline ensures that the knowledge graph is effectively constructed, semantically enriched, and adequately prepared for retrieval and evaluation in the RAG system. These steps are thoroughly explained in Chapter 5 in section 5.1.1 where an overview of the framework’s data generation component is presented.

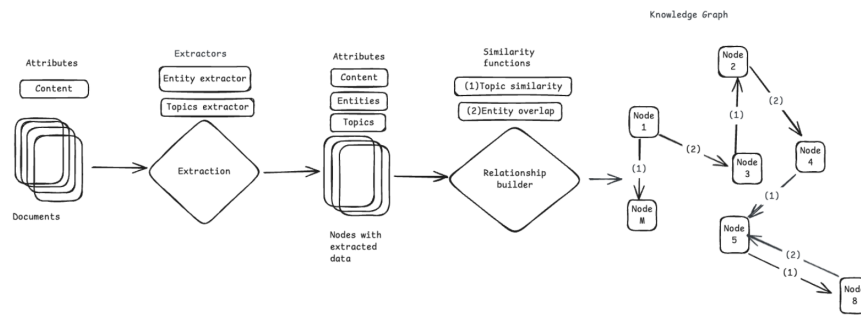


FIGURE 4.1: Knowledge Graph Creation Workflow for Synthetic Data Generation in RAGAS. (RAGAS: Automated Evaluation of Retrieval Augmented Generation, Es et al., 2023).

Overall, this chapter outlined the general architecture of the RAG system designed to enhance tourism experiences, along with the key development libraries and frameworks that support its implementation. The system’s core components, including the retrieval module, response synthesis, and knowledge base, were discussed in detail, highlighting their roles in optimizing information retrieval and generation. Additionally, the methodology for synthetic test data generation using a knowledge graph-based approach was introduced. The following chapter will provide an in-depth exploration of the system’s detailed implementation, covering the technical aspects of its components and their integration.

Chapter 5

TouaRAG Core Components & Implementation

In this chapter the RAG architectures and main components of the RAG system for tourism enhancement as outlined in Chapter 4, as well as the libraries used for their implementation, will be discussed in detail.

5.1 Framework Components

The RAG framework presented, is composed of 5 individual component modules: The Data Generation module, the Configuration and Utilization module, the Prompt Engineering and Personalization module, the LLM Querying module and the Evaluation module.

5.1.1 Data Generation

As explained in section 4.3 RAGAS library was used primarily for the data generation process in this module. From the perspective of the framework, initially data are loaded, in this specific case PDF files which are fed to a parser in order to transform them into structured information with the appropriate metadata such as page number, total pages, file path etc. The specific parser that is used throughout the thesis is the PyMuPDF ¹ parser which is a high-performance Python library for data extraction, analysis, conversion and manipulation of PDF (and other) documents. After the document parsing, the module proceeds with the knowledge graph generation according to the input files. This is done with the help of the knowledge graph transforms which help find nodes, relationships and other metadata for identified nodes

¹PyMuPDF: A high-performance Python library for PDF document processing. Available at: <https://pymupdf.readthedocs.io/>

thus building the appropriate graph. Those transforms that are applied, differ according to the overall size of the pages of the whole document provided. More specifically if more than 25% of each page within the document include "501-100000" tokens then the following transforms are used:

- **Headlines Extractor:** Queries the LLM to provide a headline for a given page content.
- **Chunk Splitter:** Splits input document pages into smaller or larger chunks according to parameters. The default parameters provided are minimum tokens per chunk to be 100 and maximum tokens per chunk to be 1000.
- **Summary Extractor:** It queries the LLM to provide a summary for each chunk's text which must be less than 10 sentences.
- **Themes Extractor:** Queries the LLM to extract the main themes and concepts from the given chunk text.
- **NER Extractor:** Queries the LLM to extract named entities from given chunk text with a parameter that specifies the maximum number of entities extracted.
- **Summary Embedding Extractor:** It takes as input the summary created from the Summary Extractor transform and then extracts its embedding from the predefined embedding model.
- **Summary Cosine Similarity Builder:** Accepts as inputs the embedding vectors from previous transform and then calculates the cosine similarity between all vector pairs. Afterwards keeps only pairs that are similar above 90% after applying L2 normalization to all vectors. To those similar pairs a relationship is created in the graph of type "cosine similarity".
- **Overlap Score Builder:** This transform takes as input the entity texts of each node. Then by iterating to all nodes it identifies entities which are repeatedly seen in all nodes and filters them out as "noisy". Afterwards for each pair of node it takes the entity strings (excluding the "noisy" entities), and the transform calculates on those the Jarro Winkler similarity distance. If the similarity is above 90% it creates a relationship between those two nodes of type "overlapped items".
- **Scoring Evaluation Filtering:** Queries the LLM to score each node content given the provided summary and text within the range 1-5. If the returned score is equal or below 2 then the node is filtered out from the KG.

If the above criteria are not met but there is a 25% or more of pages within the document that have "101-500" tokens then only Summary Extractor, Scoring Evaluation Filtering, Summary

Embedding Extractor, Themes Extractor, NER Extractor and Overlap Score Builder are used. After all those transforms the KG is completed and ready to be used for the actual synthetic data generation. The distribution of the query types explained in section 4.3 is calculated with the formula provided below:

$$\text{Query Type Distribution} = \left\{ \left(q_i, \frac{1}{N} \right) : q_i \in Q_{\text{available}} \right\},$$

where:

- $Q_{\text{default}} = \{\text{Single Hop Specific Query, Multi Hop Abstract Query, Multi Hop Specific Query}\},$
- $Q_{\text{available}} = \{q \in Q_{\text{default}} \mid q.\text{get_node_clusters}(\text{kg})\},$
- $N = |Q_{\text{available}}|.$

Here, Q_{default} represents the set of default query types. $Q_{\text{available}}$ is the subset of Q_{default} where each query q has node clusters in the provided knowledge graph (kg). The distribution assigns an equal weight of $\frac{1}{N}$ to each available query, where N is the total number of queries in $Q_{\text{available}}$. This ensures that all valid queries are treated uniformly in the distribution.

Additionally, the initialization of Persona profile is performed, which is utilized during LLM querying at data generation process, with the following characteristics:

- **Persona Name:** Traveler
- **Role Description:** Don't know much about what places to visit, where they are and their history. Wants to learn more information to improve the trip with as many details as possible.

The next step is to generate all possible scenarios for the query generation. The process solely depends on each query type used such as Single Hop Specific Query, Multi Hop Abstract Query and Multi Hop Specific Query.

For the Single Hop Specific Query type, the process begins by identifying chunk nodes containing entities. The number of samples to be created per node is then calculated to ensure that the total generated samples reach the desired number. Next, entities associated with each chunk node in the Knowledge Graph (KG) are identified. Personas are then mapped to these entities to formulate queries. Once the nodes, entities, personas, style, and length are determined, all possible combinations of these elements are created as base scenarios. Finally, sampling is performed on these base scenarios to ensure an even distribution of scenarios across nodes,

aligning with the desired number of generated data samples. As for the Multi Hop Abstract Query type, the only difference is that the process starts by identifying indirect clusters of nodes based on relationship conditions. All other steps are identical to the Single Hop Specific Query type. Finally, the Multi Hop Specific Query type, the process first filters the KG to identify clusters of nodes with a defined relation type. The required number of samples per cluster is then determined. Entities common between the nodes in the cluster are identified, followed by the association of personas with these entities to create queries. All possible combinations of nodes, entities, personas, style, and length are generated to form base scenarios. Lastly, sampling is conducted to ensure that equal scenarios are selected from each triplet created, aligning with the required number of generated data samples.

After generating scenarios for all query types, the component proceeds with data generation using LLM prompting, ensuring that each scenario is developed according to its defined properties. This section directly supports the "Domain-specific Evaluation and Dataset Creation" contribution in the data generation aspect, in Section 1.3.

5.1.2 Configuration and Utilization

For seamless integration of the different libraries used throughout this thesis, a configuration component was created to integrate each specific tool and to ensure that is flexible, scalable, and capable of handling different volumes of data with minimal manual intervention. Includes tools for preprocessing documents, managing data storage, configuring external integrations, and offering utility functions that streamline various tasks. In terms of preprocessing functionality, this component handles raw input documents for further processing by transforming them into a format suitable for embedding and querying. This involves dividing lengthy documents into smaller, contextually consistent chunks to ensure efficient handling during retrieval and embedding stages. Preprocessing also integrates additional transformations such as sentence splitting and entity extraction, which refine the data and enhance the quality of downstream operations.

Especially, entity extraction plays a crucial role in the metadata enrichment process within our retrieval-augmented generation (RAG) pipeline. To achieve effective extraction of entities, we employ the EntityExtractor module provided by LlamaIndex, which integrates with the SpanMarker library. Specifically, the pretrained model tomaarsen/span-marker-mbert-base-multinerd² is utilized, which is designed for multilingual Named Entity Recognition (NER) and at its core, the model employs the bert-base-multilingual-cased encoder, leveraging its ability to process text in a variety of linguistic contexts. The extracted entities are subsequently stored

²SpanMarker mBERT-base MultiNERD: A multilingual Named Entity Recognition model developed by Tom Aarsen. Available at: <https://huggingface.co/tomaarsen/span-marker-mbert-base-multinerd>

in the metadata field associated with each node, thereby enhancing the retrieval process by allowing for more structured and semantically meaningful search capabilities.

The storage functionality of the module provides a mechanism for managing data embeddings and storing them in a structured format, such as a vector database, ensuring that the dense and sparse representations of documents, which are critical for hybrid retrieval, are organized and readily accessible. Moreover, the use of a vector database allows the system to perform efficient similarity searches and rank retrieval results based on semantic relevance. Storage operations are closely tied to the preprocessing stage, as the preprocessed data serves as input to generate embeddings that are then stored and indexed and this is the reason those two functionalities are included in the same component. It also facilitates updates and retrieval, maintaining a seamless interaction between data persistence and runtime queries. The actual configuration subcomponent is responsible for managing the overall settings of the application. It proceeds with loading and parsing configuration files, which define paths, API keys, database connection parameters, and other application-wide settings. This ensures consistency and flexibility, as changes to system behavior can be made by simply updating configuration files. Additionally, it validates these settings to prevent runtime errors and maintains default values for critical parameters, ensuring robust operation even in the absence of explicit definitions. In addition to the configuration subsystem, also the utilization subcomponent is responsible for performing routine operations through the provision of reusable tools. These include clearing memory, handling file paths, and managing logs, among other tasks thus integrating smoothly various processes. For instance, they ensure that data flows from preprocessing to storage without interruption, handle errors gracefully, and maintain detailed logs for debugging and analysis. The workflow within this module begins with the configuration subsystem, which initializes the environment by loading the predefined required settings. Once the environment is set up, the preprocessing system ingests raw documents and applies transformations to prepare them for storage. The processed data is passed later on to the storage layer, where embeddings are generated and stored in a vector database. Throughout this process, utility functions monitor and log operations, ensuring that the workflow is both efficient and transparent. This integrated design allows the Configuration and Utilization Component to support the broader objectives of the RAG system, providing a reliable backbone for data processing, storage, and retrieval operations.

5.1.3 Prompt Engineering and Personalization

A vital part of the RAG system presented in this thesis is the Prompt Engineering and Personalization component, focusing on the interaction between users and the system by generating context-aware, personalized responses. This component leverages the flexibility of language models to provide highly specific and user-oriented outputs by utilizing customizable prompt

templates. These templates act as the bridge between raw user input and the system's capacity to generate meaningful responses, effectively guiding the language model to deliver outputs that align with user expectations and application requirements. Essentially it allows, to define and manage various templates that structure the interaction process. Each template consists of predefined instructions and placeholders for dynamic content, such as user queries or other information. For instance, a template may specify that responses must adopt the tone and style of a knowledgeable tour agent, ensuring that the system maintains a consistent narrative voice. Personalization is achieved by incorporating user-specific attributes—such as age, nationality, travel preferences, and accessibility needs—into the templates. These attributes are either preloaded or dynamically injected into the templates during runtime, tailoring the system's output to the user's profile without explicitly referencing the details in the response.

```

DEFAULT_TOUR_PROFILE = {
    "age_range": "Young Adult",
    "nationality": "Greek",
    "travel_group": "Solo Traveler",
    "purpose_of_travel": "Leisure",
    "travel_style": "Budget",
    "travel_frequency": "First-time Visitor",
    "comfort_with_local_culture": "Seeks Familiar Experiences",
    "preferred_activities": "Sightseeing",
    "accessibility_needs": "None",
    "accommodation_style_preference": "Hotel",
    "length_of_stay": "Weekend",
    "preferred_season_or_climate": "Summer",
    "social_media_and_photography": "Interest in Instagrammable Spots",
    "local_interaction": "Comfortable with Local Language",
    "interest_in_meeting_locals": "High"
}

```

LISTING 5.1: Example of a Personalized Attributes Template with Default Profile

This seamless integration of personalization ensures that the responses remain relevant and engaging while retaining a professional and concise tone. The workflow within this component begins with the selection or initialization of a suitable template based on the context of the query and the user profile. When performing evaluations without real user data, the default profile is automatically applied to populate the template. This profile includes the aforementioned attributes such as age, nationality, travel group, purpose of travel, and preferences, designed to simulate a realistic but standardized user. The filled template is then combined

with the query and any relevant contextual information retrieved from the knowledge base, within the prompt provided to the LLM, to generate a well-informed, cohesive response.

For instance, the default profile might describe a solo traveler with a budget-conscious approach who prefers cultural experiences and local cuisine. This simulated persona is consistently used to evaluate the outputs of various RAG system architectures, ensuring that differences in performance are attributed to the system's design rather than inconsistencies in the input data. By structuring the interaction through predefined templates, this component ensures that the system's responses are not only coherent but also adhere to the domain-specific requirements of the application. The use of the default profile enhances the scalability and repeatability of evaluations by providing a consistent test case that can be used across different experimental setups. This allows for a fair comparison of different RAG pipelines while maintaining the modularity needed to adapt templates for real-world user data in future explorations. Furthermore, the modular design of the templates allows for easy updates and extensions, enabling the system to adapt to new use cases or changes in user behavior without requiring fundamental architectural modifications. This adaptability, combined with the precise personalization offered by dynamic template filling and the consistency ensured by the default profile, positions this component as a cornerstone of the RAG system's ability to deliver high-quality, user-centered interactions.

```

PERSONALIZED_TOUR_AGENT_TMPL = (
    "Context information is below.\n"
    "-----\n"
    "{context_str}\n"
    "Given the context information and not prior knowledge, "
    "answer the query.\n"
    "-----\n"
    "Imagine you are the person with the below profile:\n"
    "Age Range: {age_range}, Nationality: {nationality}, Travel Group
: {travel_group}, "
    "Purpose of Travel: {purpose_of_travel}, Travel Style: {travel_
style}, Travel Frequency: {travel_frequency}, "
    "Comfort with Local Culture: {comfort_with_local_culture},
Preferred Activities: {preferred_activities}, "
    "Accessibility Needs: {accessibility_needs}, Accommodation Style
Preference: {accommodation_style_preference}, "
    "Length of Stay: {length_of_stay}, Preferred Season or Climate: {
preferred_season_or_climate}, "
    "Social Media and Photography: {social_media_and_photography},
Local Interaction: {local_interaction}, "
    "Interest in Meeting Locals: {interest_in_meeting_locals}\n"
    "-----\n"
    "Query: {query_str} \n"
    "You must carefully consider the profile in your suggestions but
you don't need to mention it when speaking.\n"
    "The answer MUST be maximum 350 words long and MUST be concise
and MUST be final.\n"
    "Answer: "
)

```

LISTING 5.2: Example of the Personalized Prompt Template

5.1.4 LLM Querying

The LLM Querying component is responsible for the interaction of the system and the underlying language models in order to generate responses to queries performed by users. This component integrates various subsystems to support multiple querying strategies, uses both dense and sparse retrieval techniques and seamlessly combining them with powerful language models from providers like Hugging Face and OpenAI. For each provider, the module uses a

specific model by default. For HuggingFace the "Llama-3.2-3B-Instruct" ³ model with quantization configuration is selected and for the OpenAI provider ⁴ the default selection is the "gpt-4o-mini". The quantization configuration essentially, lowers the precision of the model weights (in this case to 16-bit floating point) for execution time reduction and performance reasons. In terms of embedding models the default HuggingFace model is the "BGE-M3" [5] and for OpenAI the "text-embedding-3-small" model is chosen. Moreover this module provides helper functions for seamless integration between provider APIs and allows easy addition of any model desired from the two aforementioned providers. A significant aspect of this component is also its adaptability to different system architectures, such as baseline, automerger, hybrid vectors, or graph-based methods like graph-ms-local which will be explained in detail in section 5.2. The integration of diverse querying strategies, the use of advanced language models, and the adaptability to different architectures make this component a cornerstone of the RAG framework implemented in this thesis.

5.1.5 Evaluation

Responsible for the performance evaluation and assessment of the various system architectures used in the framework presented in this thesis, is the Evaluation component. It ensures that the responses generated by the system meet high standards of quality, relevance, and factual accuracy both in terms of the actual answer as well as in terms of the information retrieved whether it was actually relevant or not. By providing measurable outcomes, this component not only validates the effectiveness of the system but also identifies areas for improvement, guiding the iterative development process. The primary purpose of this component is to evaluate the outputs of the system using predefined metrics that align with the goals of the application. These metrics include measures of factual correctness, semantic similarity, relevancy, and faithfulness, among others which will be thoroughly explained in Section 6.3. The evaluation workflow begins by loading a dataset that simulates user interactions. This dataset typically consists of user queries, reference contexts, and expected answers. Each query is passed through the RAG system, which retrieves relevant documents, generates a response, and retrieves the contexts that informed that response. The evaluator then compares the system-generated response with the expected answer using a set of defined metrics. This comparison is not limited to the surface-level similarity but delves deeper into the semantic alignment and the contextual accuracy of the responses. For instance, semantic similarity ensures that even if the phrasing of the response differs, the meaning remains aligned with the reference. An essential aspect of

³Llama-3.2-3B-Instruct: A multilingual large language model developed by Meta AI, optimized for multilingual dialogue and summarization tasks. Available at: <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>

⁴OpenAI API: A comprehensive interface for accessing OpenAI's language models, enabling tasks such as text generation, translation, and summarization. Available at: <https://platform.openai.com/docs/api-reference/introduction>

the evaluation process is its ability to handle various system configurations. This flexibility allows the system to test different RAG pipelines and architectures under consistent conditions, providing a fair comparison of their performance. The evaluator achieves this by maintaining a modular design where the metrics and evaluation criteria are independent of the underlying system architecture. This modularity ensures that the same dataset and metrics can be used to evaluate systems ranging from basic retrieval-augmented generation pipelines to more complex graph-based approaches. Furthermore critical feature of the Evaluation component is its ability to generate and save samples during the evaluation process. These samples capture the details of each evaluation instance, including the query, the retrieved contexts, the generated response, and the reference answer. By saving these samples, the system enables detailed post-hoc analysis, allowing developers to investigate specific failures or patterns in the system's performance. Furthermore, this feature ensures reproducibility, as the saved samples can be reused to validate future iterations of the system. The evaluation process is designed to handle large datasets efficiently. It uses parallel processing and optimized workflows to evaluate multiple queries simultaneously, ensuring scalability and responsiveness. In addition, it incorporates cost analysis to track token usage during evaluations, providing insights into the computational efficiency of the system. This feature is particularly valuable in scenarios where the system relies on commercial APIs like OpenAI for language model interactions, as it helps in managing operational costs. The evaluation component detailed in this section, supports the "Domain-specific Evaluation and Dataset Creation" contribution in the evaluation aspect, in Section 1.3.

5.2 Implemented RAG Architectures

In this thesis 5 different RAG architectures were tested on tourism domain specific data. A basic architecture was selected as a baseline and the others implemented according to past literature, were compared to identify which architectures can work best with data in this specific domain and for the intended use case of enhancing the tourism experience.

5.2.1 Baseline

The baseline architecture within the Retrieval-Augmented Generation (RAG) application is built to offer a clear, effective, and robust foundation for retrieving and processing information. It employs the BM25 [26] retrieval algorithm, which uses a probabilistic approach to rank documents based on their relevance to a query. BM25 is particularly effective for text-centric datasets because of its ability to balance term frequency and inverse document frequency with query-document length normalization, and detailed explanation of the algorithm can be found in Section 3.1.1.1 . This method ensures that retrieved results are relevant accurate, where

precise and informative responses are critical. The baseline architecture starts with initializing the necessary language model and settings. Afterwards it applies specific transformations to ensure the input data is optimized for retrieval and generation. These transformations include chunk splitting to divide content into manageable chunks of size 400 tokens with overlapping of maximum 50 tokens and entity extraction to identify key entities within the text. Chunk splitting ensures that each chunk remains within a predefined size limit, facilitating seamless integration with the retriever and generative components. More specifically an implementation from Llamaindex ⁵ was used which avoids to split specific parts of sentences or paragraphs to ensure coherence. Entity extraction aids in enriching the document metadata, which can be crucial for downstream tasks and providing more context to our answer generating LLM. For entity extraction, a readily available implementation from Llamaindex library was used which also offered the possibility to execute the calculations in GPU. All details are provided in previous section regarding metadata enrichment with NER extraction 5.1.2. The document preprocessing pipeline involves parsing the data from source documents, applying the aforementioned transformations, and converting the resulting chunks into a structured format known as nodes. These nodes are stored in a document store that serves as the primary index for retrieval. The BM25 retriever is initialized using these nodes. It tokenizes the document content, removes stopwords, applies stemming, and builds an index that enables efficient retrieval based on query similarity. During query execution, the user's input is preprocessed to match the format of the indexed nodes. The BM25 retriever calculates similarity scores between the query and the indexed nodes, returning the top-ranked results. These results are passed through a query engine that applies a personalized prompt template. The template guides the language model to synthesize a response tailored to the specific domain this RAG system is intended for, the tourism domain. Once the top relevant chunks are retrieved, the system begins by using the highest-ranked chunk as context for the language model to generate an initial response. The response is then fed back into the query engine, alongside the next retrieved chunk, forming a cumulative context. The LLM is then re-invoked with this expanded input, which includes the user's original query, the previously generated answer, and the additional context from the new chunk. This iterative process continues until all retrieved chunks have been processed. Each iteration refines the response, ensuring that the final output reflects the combined knowledge contained within all relevant chunks. This way, potential limitations in a single-pass approach are addressed, where the language model might not effectively synthesize information from multiple chunks simultaneously. By incrementally incorporating additional context, the architecture ensures that the response remains coherent and progressively enriched with new, relevant information. It effectively mitigates issues of information loss or dilution, which can occur when handling extensive input data in one step.

⁵LlamaIndex: An open-source data framework for building applications with large language models, developed by the LlamaIndex community. Available at: https://github.com/run-llama/llama_index

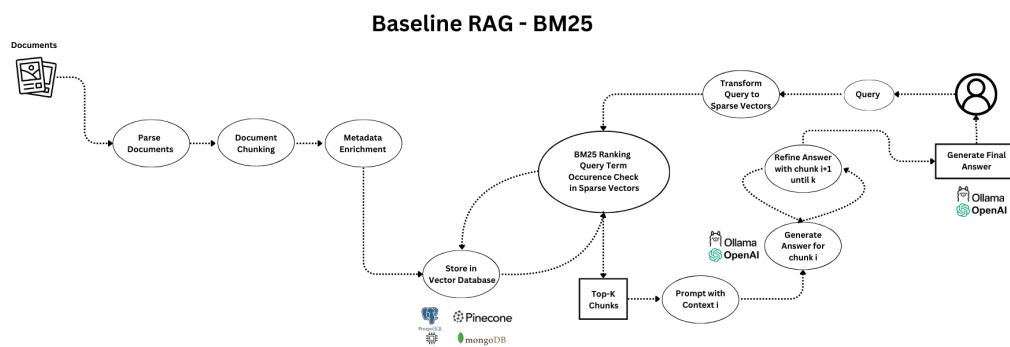


FIGURE 5.1: Baseline Architecture Workflow Diagram

5.2.2 Hybrid Vectors

The Hybrid Vectors architecture combines sparse and dense vector representations to improve the effectiveness of IR in the context of RAG systems. This approach is used to take advantage of both methodologies strengths. Sparse representations, in this case BM25, excel in leveraging exact matches between query terms and document tokens. Dense representations, on the other hand, are derived from embeddings produced by transformer models and excel in capturing semantic similarities, even between text fragments that do not share explicit vocabulary. Moreover while Sparse representations are efficient and interpretable, they may fail to capture nuances of a query especially when synonyms and paraphrased expressions are used. To solve those issues the Dense embedding representations, map semantically similar phrases into close proximity within a high-dimensional vector space. But Dense representations also have some downsides like, struggling with rare or domain-specific terms that are well handled by sparse methodologies. The workflow of this architecture is structured around several key stages. First, like the previous approach, the documents intended for retrieval are preprocessed. This pre-processing involves the same steps as in Baseline methodology, splitting the documents into manageable chunks, often based on sentence boundaries, and then NER is another vital pre-processing step. These entities can enrich the metadata associated with each chunk, aiding in retrieval by providing additional structured context. Afterwards once the chunks are prepared, they are processed to generate dense embeddings using a transformer-based embedding model. Each chunk is transformed into a high-dimensional vector that captures its semantic content. These embeddings are then stored in a vector database, enabling efficient similarity-based retrieval during the query phase. Concurrently, sparse representations are created using

BM25, which indexes the tokenized text chunks to allow for exact and near-exact term matching. When a query is submitted, it is first processed through both the sparse and dense retrieval systems independently. The sparse retrieval methodology applies BM25 to identify text fragments that contain exact or nearly exact matches with the query terms. Simultaneously, the dense methodology generates an embedding of the query using the same model employed during document preprocessing and calculates its similarity to the precomputed document embeddings. Each retrieval method generates its own ranked list of relevant chunks based on their respective criteria. The ranked results from these two retrieval pipelines are then fused into a unified ranking. To ensure robustness of retrieved results, the implementation that `llmaindex` provides, performs augmentation of the original user query and generates 3 more queries (in total 4) from which the results extracted will be selected and fused. This fusion process can involve techniques such as reciprocal rank fusion, weighted scoring, or relative score normalization to combine the outputs in a way that reflects the strengths of both methods. In this thesis and for evaluating the architecture we use the RRF method. In detail, RRF calculates a combined score for each document using the formula:

$$\text{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}$$

where $r(d)$ is the rank of a document d from a retriever r , R is the set of retrievers, and k (commonly 60) smooths contributions from lower-ranked documents. This formula prioritizes documents highly ranked by multiple retrievers while diminishing the influence of lower-ranked ones. The unified ranking, created by sorting documents based on their RRF scores, balances lexical precision and semantic depth. The top- k results are then passed to the query engine for iterative answer generation, ensuring a robust response with reduced risk of hallucination and improved contextual relevance. Like in previous approach, generation is an iterative process that seeks to integrate all retrieved chunks into a cohesive final response. Initially, the query engine sends the highest-ranking chunk to the language model to generate a preliminary answer. The generated answer is then used as part of the context for the next query iteration, where it is combined with the subsequent chunk. This iterative refinement continues until all retrieved chunks have been processed, ensuring that the final response is informed by the full breadth of the retrieved information.

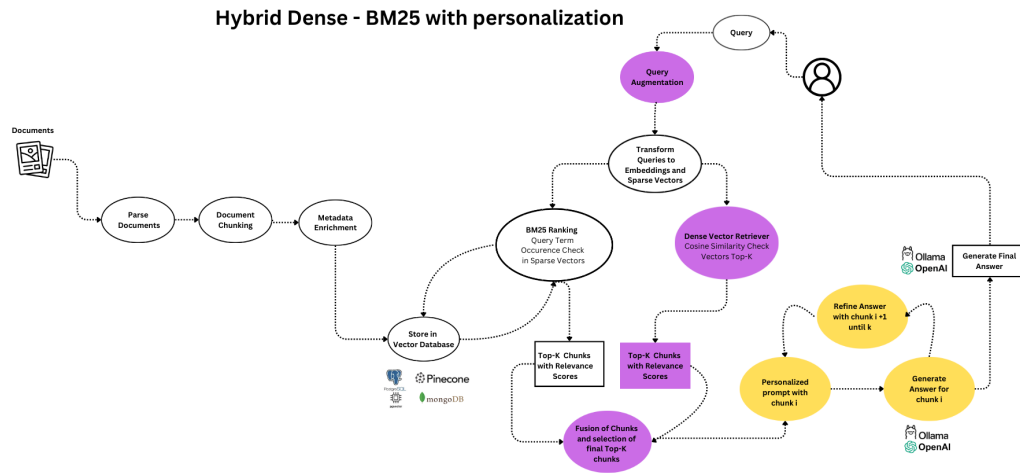


FIGURE 5.2: Hybrid Vectors Architecture Workflow Diagram

5.2.3 AutoMerge

The AutoMerge architecture is designed to optimize the synthesis of answers by iteratively merging and refining information from retrieved textual chunks and is primarily based on the AutoMerging Retriever implementation provided by the llamaindex library. Its primary function lies in ensuring that information synthesized from different parts of a dataset is contextually coherent and detailed. This is achieved by leveraging a structured hierarchy of document nodes, combining fine-grained data from the leaf nodes into a more comprehensive representation, and iteratively querying the language model for progressively refined answers. The workflow of AutoMerge begins with parsing and preprocessing input documents. Documents are parsed into a hierarchical structure of nodes, where each level corresponds to progressively smaller chunks of data. For example, the top-most nodes represent larger sections, while leaf nodes are the most granular pieces of information. This hierarchical setup is created using a series of transformations, such as chunking by sentence size, identifying entities, and embedding representations. These transformations not only structure the data but also enhance its relevance for downstream retrieval tasks. In this thesis the leaf level is chunked with 128 tokens per chunk, then the middle level is split with 512 tokens per chunk and finally the parent level is split with 2048 tokens per chunk. Once the hierarchy is established, the leaf nodes are indexed in a vector database in this case PGVector to facilitate similarity-based retrieval. When a query is issued, the system retrieves a predefined number of top-k relevant nodes using vector similarity. The AutoMerge retrieval methodology then evaluates these retrieved chunks to identify opportunities for consolidation. Specifically, it examines whether multiple chunks refer to a common parent node and calculates a similarity ratio between them. If this ratio

exceeds a defined threshold, the chunks are merged into the parent node, effectively aggregating their content. After the merging is finished, the retrieved results are used to generate a preliminary answer via the corresponding language model. This answer, combined with additional retrieved chunks, forms the input for subsequent queries to the language model. This iterative process continues until all relevant chunks have been incorporated, ensuring that the generated answers are comprehensive and contextually enriched. The hierarchical nature of the node structure and the merging process enable the system to condense a large volume of potentially redundant or disjointed information into a coherent response. Overall, AutoMerge architecture has the ability to dynamically adjust the context presented to the language model. By merging related nodes it minimizes information overload while preserving essential details. This ensures that the language model receives a concise yet informative context, leading to more accurate and meaningful outputs. Additionally, the integration of parent-child relationships between nodes facilitates a natural flow of information, where higher-level summaries are informed by their constituent details. This iterative, context-aware approach is particularly advantageous for applications where nuanced understanding and detailed synthesis are required, such as tourism, where complex queries often span multiple facets of a destination or service.

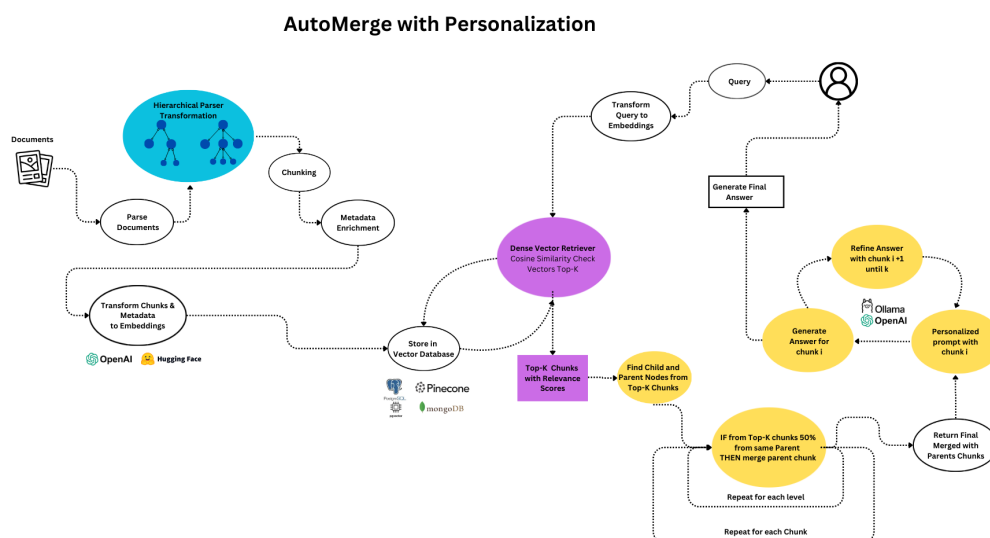


FIGURE 5.3: AutoMerge Architecture Workflow Diagram

5.2.4 HyDE

The HyDE (Hypothetical Document Embeddings) architecture is a novel methodology designed to enhance the capabilities of zero-shot dense retrieval in information retrieval systems. Its primary innovation lies in leveraging LLMs to generate hypothetical documents in response

to user queries. These generated documents serve as intermediaries, representing potential answers and facilitating retrieval by encoding them into embeddings. Unlike traditional retrieval approaches that rely heavily on relevance labels and supervised learning, HyDE circumvents the need for explicit training on labeled datasets, making it particularly effective in scenarios where such labels are scarce or unavailable. The operation of HyDE begins with a natural language query posed by the user. Instead of directly searching the corpus, the system utilizes an LLM trained to follow instructions to generate a plausible, hypothetical document that might answer the query. This document is not necessarily factually accurate or grounded in the corpus but serves as a meaningful approximation of a relevant response. The generated text captures relevance patterns and contextual cues, which are encoded into dense embeddings using a pre-trained, unsupervised contrastive encoder. The embeddings of the hypothetical document are then employed to identify similar real documents in the corpus based on vector similarity. After the top-k most relevant chunks of text are retrieved based on their similarity to the hypothetical embedding, the first chunk is used to generate an initial answer with the LLM. This answer, combined with the new context from the subsequent chunk, is provided back to the LLM along with the original query. The process repeats, incorporating additional retrieved chunks iteratively, ensuring that the final response integrates information from all relevant parts of the corpus. This iterative querying strategy not only refines the answer but also allows for dynamic adjustment to the retrieved content, progressively improving alignment with the user's query. HyDE's architecture is underpinned by a carefully orchestrated interplay between generative and contrastive components. The generative LLM acts as a flexible and powerful mechanism for relevance modeling, offloading the complex task of capturing user intent and generating appropriate hypothetical documents. The contrastive encoder, on the other hand, serves as a robust filtering mechanism, compressing the generated content into dense vectors and eliminating extraneous details that do not contribute to document similarity. The practical workflow of HyDE involves multiple stages. First, documents in the corpus are pre-processed and encoded into embeddings using the contrastive encoder, creating a searchable vector space. When a query is issued, the system generates a hypothetical document using the LLM, informed by a custom prompt template designed to align the generated content with the nature of the query. This document is then embedded and used to query the vector space, retrieving the most similar documents. The iterative querying and refinement loop follows, utilizing the retrieved chunks to iteratively synthesize and improve the final answer.

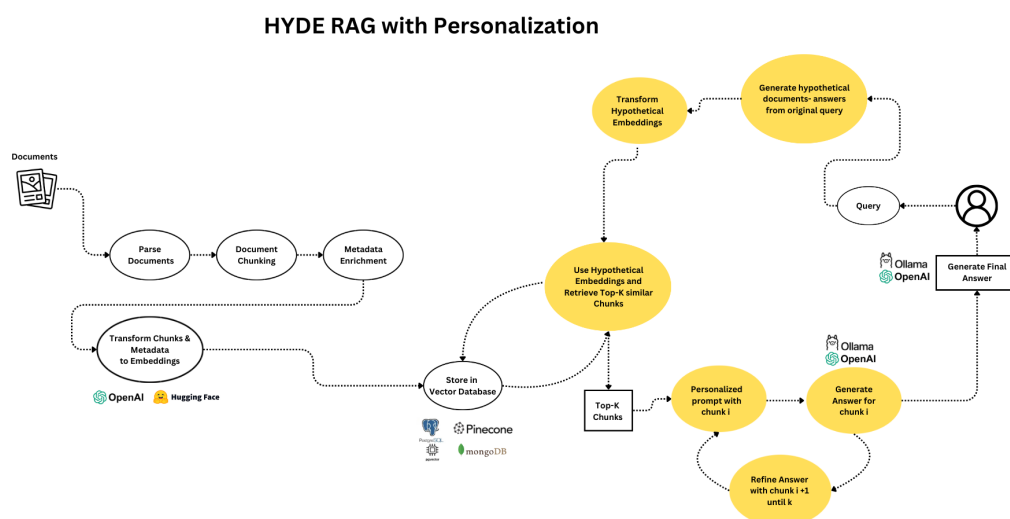


FIGURE 5.4: HyDE Architecture Workflow Diagram

5.2.5 Local GraphRAG

The GraphRAG architecture represents a sophisticated approach within the Retrieval-Augmented Generation (RAG) paradigm, emphasizing a hierarchical, graph-based indexing methodology to improve information retrieval and synthesis from large, structured, or semi-structured datasets. In contrast to traditional RAG systems relying on vector similarity-based retrieval of textual chunks, GraphRAG integrates knowledge graph construction to establish more meaningful connections between distant data points. This method is particularly advantageous in contexts requiring reasoning over complex datasets, where insights arise from the relationships and attributes of entities rather than isolated textual fragments. The core of GraphRAG's functionality is its ability to construct a graph-based index from a corpus, where the nodes represent entities (e.g., people, places, or concepts), and edges denote relationships or interactions between these entities. This index is built through multiple stages. Initially, the source text is preprocessed into manageable units, after which entities, relationships, and key claims are extracted using language models. These graph elements are further enhanced with attributes such as temporal details or context-specific claims, enriching the representational quality of the graph. A community detection algorithm, such as Leiden, partitions the graph into modular communities, uncovering hierarchical structures within the data. During indexing, the hierarchical clustering allows the generation of summaries at various levels of granularity, ranging from local details within sub-communities to broader overviews of the dataset's overarching themes. These summaries serve a dual purpose: they provide a structured understanding of the dataset for exploratory analysis and form the foundation for efficient querying. When a query is posed, the system adapts its response strategy based on the query's scope. For local queries

which are the only ones used in this thesis, focusing on specific entities, the graph's community structure is leveraged to retrieve directly relevant nodes and their immediate relationships. For broader, global queries, the hierarchical summaries are employed in a map-reduce fashion, aggregating relevant information from sub-communities to produce a comprehensive answer. This approach ensures that responses maintain both specificity and contextual breadth, aligning with the nuanced requirements of query-focused summarization tasks. Unfortunately global and drift type of queries were not included due to high costs of querying the LLM providers. GraphRAG's local search functionality exemplifies its efficiency and relevance in addressing specific informational needs. By utilizing pre-generated summaries and leveraging the graph's topology, the system minimizes computational overhead while maintaining high relevance and accuracy. This is particularly significant for domains like tourism, where users may seek detailed insights into localized attractions, relationships between cultural or historical sites, or even aggregated reviews and narratives connected to a particular theme. The structured pipeline of GraphRAG embodies the principles of modularity, scalability, and adaptability. Its ability to extract meaningful knowledge from vast datasets and organize it into an accessible, query-ready format represents a significant evolution in RAG systems. By integrating advanced graph analytics with language models, GraphRAG not only addresses the limitations of traditional RAG methods in handling complex, interconnected datasets but also opens new pathways for knowledge discovery and enhanced user interactions in specialized domains. This architecture, while computationally intensive during the indexing phase, proves resource-efficient and robust in its operational querying phase, offering a balanced trade-off for applications requiring iterative and in-depth exploration of datasets. It is important noting that, in this thesis, due to the inability to integrate llamaindex with this approach (constraint from GraphRAG library) several aspects discussed previously like the preprocessing (e.g. meta-data enrichment), the vector stores (e.g. PostgreSQL is not used) and the query methodology are different.

Overall, this chapter has provided an in-depth analysis of the core components and architectures of the RAG system implemented for tourism experience enhancement. The combination of all the implemented components, along with the architectures selected to incorporate in the framework, align with the "Development of an Open Source End-to-End RAG Framework" contribution, mentioned in Chapter 1 (see Section 1.3). With the implementation details established, the next chapter will focus on the experimentation process, where the effectiveness of different RAG architectures is tested. The evaluation methodology, experimental setup, and key performance metrics used to assess the system, will be presented in detail in Chapter 6 , offering insights into the comparative strengths and weaknesses of each approach.

Chapter 6

Experimentation & Validation

This chapter presents the experimentation and validation process undertaken to evaluate the five Retrieval-Augmented Generation (RAG) architectures implemented in this thesis: Baseline, Hybrid Vectors, AutoMerge, HyDE, and GraphRAG.

6.1 Overview

The goal of this experimentation is to measure the effectiveness, efficiency, and domain-specific suitability of these architectures in enhancing the tourism experience. The experimentation framework implemented in this thesis employs a systematic approach to assess retrieval quality, generative accuracy, and the integration of both components across different RAG configurations. A synthetic dataset specifically tailored for tourism was generated using advanced preprocessing and knowledge graph enrichment techniques. The dataset includes a variety of query types, such as single-hop, multi-hop, and abstract queries, to simulate diverse real-world user interactions as explained in section 4.3. Each architecture was tested under consistent conditions using identical dataset, same parameters, evaluation metrics and computational resources to ensure a fair comparison. Several metrics such as factual correctness, semantic similarity, and relevance were utilized to quantify performance, while specialized frameworks like RAGAS provided tools and ready-to-use implementations for comprehensive evaluation. Furthermore, by analyzing the results, this chapter aims to provide actionable insights into the strengths and limitations of each architecture. These findings contribute to understanding how RAG systems can be optimized for domain-specific applications, particularly in the context of tourism, where user-centric and factually accurate responses are critical.

6.2 Experimental Setup

The experimentation procedure setup for the evaluation of the RAG system implemented in this thesis, is designed, taking into consideration the consistency of the results. In this section, the dataset, the computational environment, the evaluation metrics and the testing procedures used throughout the experimentation phase are outlined.

6.2.1 Dataset

As described in Section 5.1.1 a dataset created specifically for the tourism domain using the RAGAs library. For the dataset creation and to reduce workload, a tourist guide regarding only Thessaloniki of approximately 120 pages was used ¹. The generated dataset consists of 105 records and has size of 576 KB. Each record includes:

1. **Input Query:** Query submitted for evaluation.
2. **Reference Context:** Contexts retrieved or relevant to the query, including multiple levels of complexity (e.g., single-hop and multi-hop relationships).
3. **Reference:** The expected answer or ground truth to the query.
4. **Synthesizer Name:** The query type used to generate the response (e.g., Multi Hop Specific Query).

Example Record:

1. **Input Query:** What are some notable Byzantine monuments in Thessaloniki?
2. **Reference Context:** "UNESCO BYZANTINE MONUMENTS Thessaloniki has been one of the first towns to know the Christian religion by the Apostle Paul; during the Byzantine period it used to be the second most important city ...
3. **Reference:** "Byzantine church-building refers to the construction of Christian monuments and churches during the Byzantine period, particularly in Thessaloniki, which was the second most important city of the empire after Constantinople. This period, spanning from the 4th to the 15th century, saw the creation of significant Christian monuments that have a decisive influence on the entire Byzantine world. These monuments are recognized for their architectural and artistic value, leading UNESCO to declare them World Heritage Monuments in 1988."

¹Visit Greece, Thessaloniki Guide. Available at: <https://www.visitgreece.gr/mainland/macedonia/thessaloniki/>

4. **Synthesizer Name:** Single Hop Specific Query

This dataset represents a balanced distribution of query types, enabling the evaluation of RAG architectures on both straightforward and complex information retrieval tasks.

6.2.2 Computational Environment

The experiments were conducted on a budget-oriented machine equipped with an AMD Ryzen 5 3600 processor featuring six cores, an NVIDIA GeForce RTX 3060 graphics card with 12 GB of memory, and 16 GB of DDR4 RAM. On the software side, the implementation utilized Python 3.11.10 as the primary programming language. Several key libraries and frameworks were employed, including LlamaIndex², Hugging Face Transformers³, OpenAI API⁴, PostgreSQL with PGVector⁵, GraphRAG MS [7], and RAGAS [8]. The system operated on Linux 24.04.1 LTS⁶ as the underlying operating system.

From the LLM API Integrations, the default models selected for the experimentation phase were as shown in Table 6.1:

TABLE 6.1: Model Configurations

Category	LLM Model	Embedding Model
Local Models	meta-llama/Llama-3.2-3B-Instruct (Quantized, Locally loaded)	BAAI/bge-m3 (Locally loaded)
OpenAI Models	gpt-4o-mini	text-embedding-3-small
Evaluation Models	gpt-4o-mini	text-embedding-3-small

6.3 Evaluation Metrics

In order to assess the performance of the RAG system in every aspect of the RAG pipeline and evaluate whether it is efficient or not in a consistent manner, retrieval quality and generative quality metrics were used. All metrics are implemented within the RAGAS library.

²LlamaIndex: A data framework to help you build LLM applications, providing data connectors, indices, and advanced retrieval methods. Available at: https://github.com/run-llama/llama_index

³Hugging Face Transformers: A library providing general-purpose architectures for Natural Language Understanding (NLU) and Natural Language Generation (NLG) with over 32+ pretrained models in 100+ languages. Available at: <https://huggingface.co/transformers/>

⁴OpenAI API: An API providing access to advanced language models for various applications, including text generation and analysis. Available at: <https://platform.openai.com/docs/api-reference>

⁵PostgreSQL with PGVector: An open-source relational database system enhanced with the PGVector extension to support vector similarity search. PGVector available at: <https://github.com/pgvector/pgvector>

⁶Linux 24.04.1 LTS: A long-term support version of the Linux operating system, providing stability and extended support for applications. Available at: <https://releases.ubuntu.com/24.04/>

6.3.1 Retrieval Quality Metrics

The following metrics focus only on evaluating the quality of the retrieval process, ensuring that the retrieved contexts are relevant to the query submitted by the users.

6.3.1.1 Context Precision

Context precision is responsible for measuring the proportion of relevant chunks within the retrieved context by the RAG system in other words the information used by the system to generate the answer. Firstly, this metric calculates the Precision@k rank for each chunk in the retrieved context. Afterwards the Precision@k is used to calculate the ContextPrecision@K where K is the total number of retrieved chunks. In order to define whether a context chunk is relevant or not, an evaluation LLM is employed and provides insight on whether the chunk is relevant (returns value 1) or irrelevant (returns value 0). More specifically the calculation equations as defined in RAGAS documentation are provided below:

$$\text{Context Precision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\text{Total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}}$$

Where K is the total number of chunks retrieved in retrieved context and $v_k \in \{0, 1\}$ is the relevance indicator at rank k .

Below an example is provided along with its setup, showcasing the calculation of the metric:

Given a list of Relevance Indicators, pinpointing whether an item is relevant (1 for relevant, 0 for not relevant):

$$\text{Relevance Indicator List} = [1, 0, 1, 1, 0]$$

The denominator represents the total number of relevant items in the list. This is computed as the sum of all the relevant items:

$$\text{Denominator} = 1 + 0 + 1 + 1 + 0 = 3$$

- At $k = 1$:

- Precision@1 = $\frac{1}{1} = 1.0$
- Relevance Indicator[1] = 1
- Contribution = $1.0 \cdot 1 = 1.0$
- At $k = 2$:
 - Precision@2 = $\frac{1}{2} = 0.5$
 - Relevance Indicator[2] = 0
 - Contribution = $0.5 \cdot 0 = 0.0$
- At $k = 3$:
 - Precision@3 = $\frac{2}{3} \approx 0.666$
 - Relevance Indicator[3] = 1
 - Contribution = $0.666 \cdot 1 = 0.666$
- ...
- At $k = 5$:
 - Precision@5 = $\frac{3}{5} = 0.6$
 - Relevance Indicator[5] = 0
 - Contribution = $0.6 \cdot 0 = 0.0$

Summing all contributions:

$$\text{Numerator} = 1.0 + 0.0 + 0.666 + \dots + 0.0 = 2.416$$

$$\text{Context Precision} = \frac{\text{Numerator}}{\text{Denominator}}$$

Substituting the values:

$$\text{Context Precision} = \frac{2.416}{3} \approx 0.805$$

Finally there are two different modes of this metric used in this thesis that affect the way the LLM judges whether a chunk is relevant or not. The first mode is the "Context Precision without reference" mode in which the LLM relies on the generated response (regardless of its accuracy) to judge the aforementioned retrieved context by the system. The second mode is

the "Context Precision with reference" and essentially instead of using the generated response, it uses the referenced response (ground truth) designated in the test dataset to evaluate the retrieved context.

6.3.1.2 Context Recall

Context Recall measures how many of the relevant documents (or chunks) were successfully retrieved by the RAG system. It focuses on ensuring that the system does not miss important pieces of information from the relevant context. This metric is calculated based on the overlap between the retrieved contexts and the reference, with higher values indicating fewer omissions of relevant information. The Context Recall metric requires a reference to compare against which can be either reference contexts (with Non-LLM Context Recall) or reference response (with LLM Context Recall), as it evaluates how comprehensively the system retrieves relevant information. There are two variations of this metric used in the thesis. The first variation is "LLM-Based Context Recall," where an LLM is employed to analyze and break down the reference into claims and check whether each claim can be attributed to the retrieved contexts. The second variation is "Non-LLM-Based Context Recall," which uses traditional string similarity measures to evaluate the relevance of the retrieved contexts. These modes allow for flexibility depending on the use case and evaluation needs.

The calculation equation is provided below:

$$\text{Context Recall} = \frac{\text{Number of Relevant Retrieved Documents or Supported Claims}}{\text{Total Number of Relevant Documents or Claims}}$$

Below an example is provided along with its setup, showcasing the calculation of the metric:

- **User Input:** "Where is the Eiffel Tower located?"
- **Reference Contexts or Reference Response Claims from LLM:**
 - Context A: "Paris is the capital of France."
 - Context B: "The Eiffel Tower is located in Paris."
 - Context C: "The Eiffel Tower is one of the most famous landmarks in Paris."
- **Retrieved Contexts:**
 - Context 1: "Paris is the capital of France."
 - Context 2: "The Eiffel Tower is located in Paris."
 - Context 3: "The Eiffel Tower is one of the most famous landmarks in Europe."

The calculation steps are as follows:

- **Step 1: Identify Relevant Retrieved Contexts** Compare each retrieved context with references. A retrieved context is considered relevant if it matches or overlaps significantly with any of the references.
 - Context 1: Relevant (matches Context A)
 - Context 2: Relevant (matches Context B)
 - Context 3: Irrelevant (no match)
- **Step 2: Count Relevant Retrieved Contexts** Total number of relevant retrieved contexts = 2 (Context 1 and Context 2).
- **Step 3: Count Total Relevant Contexts in Reference** Total number of relevant contexts in the reference = 3 (Context A, Context B, Context C).
- **Step 4: Calculate Context Recall** Substituting the values in the Context Recall formula:

$$\text{Context Recall} = \frac{2}{3} \approx 0.666$$

6.3.1.3 Context Entity Recall

Context Entity Recall as implemented in RAGAS library measures the fraction of entities present in both the reference and the retrieved contexts relative to the total entities in the reference. This metric is particularly useful in fact-based applications such as tourism assistance or historical question answering, where the presence of specific entities (e.g., names, dates, locations) is critical. By evaluating how well the retrieved contexts cover the entities mentioned in the reference, Context Entity Recall helps assess the effectiveness of the retrieval mechanism in preserving important factual information. The metric is computed by first extracting entities from the reference (denoted as RE) and the retrieved contexts (denoted as RCE). The Context Entity Recall is then calculated as the ratio of the overlapping entities to the total entities in the reference:

$$\text{Context Entity Recall} = \frac{|RE \cap RCE|}{|RE|}$$

Below an example is provided along with its setup, showcasing the calculation of the metric:

- **User Input:** "Tell me about the White Tower of Thessaloniki."

- **Reference:** "The White Tower of Thessaloniki is a monument and museum on the waterfront of Thessaloniki, Greece. It was constructed during the Ottoman era in the 15th century and is a symbol of the city."
- **Retrieved Contexts (Case 1):** "The White Tower of Thessaloniki, located on the waterfront in Greece, is a historical monument. It was built in the 15th century during the Ottoman period and has become an iconic symbol of the city."
- **Retrieved Contexts (Case 2):** "The White Tower is a popular tourist attraction in Thessaloniki. It is located in Greece and is known for its historical significance as a museum and cultural landmark."

The calculation steps are as follows:

- **Step 1: Identify Entities in Reference** Extract entities from the reference text:

$$RE = \{\text{White Tower of Thessaloniki, Thessaloniki, Greece, 15th century, Ottoman era}\}$$

- **Step 2: Identify Entities in Retrieved Contexts** Extract entities from the retrieved contexts of each mechanism:

- **Case 1 (High Entity Recall):** $RCE_1 = \{\text{White Tower of Thessaloniki, Thessaloniki, Greece, 15th}$

- **Case 2 (Low Entity Recall):** $RCE_2 = \{\text{White Tower, Thessaloniki, Greece, museum}\}$

- **Step 3: Compute Overlapping Entities** Calculate the intersection between RE and each RCE :

- **Case 1:** $|RE \cap RCE_1| = 5$ (entities: White Tower of Thessaloniki, Thessaloniki, Greece, 15th century, Ottoman period)

- **Case 2:** $|RE \cap RCE_2| = 3$ (entities: White Tower, Thessaloniki, Greece)

- **Step 4: Calculate Context Entity Recall** Apply the formula for each mechanism:

- **Case 1:**

$$\text{Context Entity Recall} = \frac{5}{5} = 1.0$$

- **Case 2:**

$$\text{Context Entity Recall} = \frac{3}{5} = 0.6$$

6.3.2 Generative Quality Metrics

Generative Quality metrics evaluate the quality, factual accuracy, and general relevancy of the generated response based mainly on the query, reference and retrieved context.

6.3.2.1 Response Relevancy

Response Relevancy measures how closely a generated response aligns with the intent and content of the user input. This metric evaluates whether the answer directly addresses the query while avoiding incomplete or redundant information. Higher scores indicate stronger alignment, while lower scores reflect gaps or irrelevant details. The calculation involves reverse-engineering potential questions with the use of LLM, from the response and comparing their semantic similarity to the original user input.

The metric is computed as follows:

1. **Generate artificial questions:** An LLM creates n questions (default: 3) based solely on the content of the response.
2. **Compute similarity:** Calculate the cosine similarity between the embeddings of the user input (UI) and each generated question (Q_i).
3. **Average scores:** The final Response Relevancy score is the mean of these similarity values:

$$\text{Response Relevancy} = \frac{1}{n} \sum_{i=1}^n \cos(UI, Q_i)$$

Below an example is provided along with its setup, showcasing the calculation of the metric:

- **User Input:** "What are the main attractions in Thessaloniki, Greece?"
- **Low Relevancy Response:** "Thessaloniki has the White Tower and Aristotelous Square."
- **High Relevancy Response:** "Thessaloniki, Greece, is known for landmarks like the White Tower, Aristotelous Square, and the Rotunda. It also boasts vibrant markets, Byzantine walls, and the UNESCO-listed Paleochristian monuments."

The calculation steps are explained in detail below:

1. **Generate Questions from Responses:**
 - **Low Relevancy Response:** The generated questions are:
 - "What is the White Tower in Thessaloniki?"
 - "Where is Aristotelous Square located?"
 - "Name two places in Thessaloniki."
 - **High Relevancy Response:** The generated questions are:
 - "What are the historical landmarks in Thessaloniki?"

- "Which UNESCO sites exist in Thessaloniki?"
- "Describe cultural attractions in Thessaloniki, Greece."

2. **Compute Cosine Similarity:** Assume the following cosine similarity scores (hypothetical values):

- **Low Relevancy Response:**

$$\cos(UI, Q_1) = 0.7, \quad \cos(UI, Q_2) = 0.6, \quad \cos(UI, Q_3) = 0.5$$

$$\text{Response Relevancy} = \frac{0.7 + 0.6 + 0.5}{3} = 0.6$$

- **High Relevancy Response:**

$$\cos(UI, Q_1) = 0.9, \quad \cos(UI, Q_2) = 0.8, \quad \cos(UI, Q_3) = 0.85$$

$$\text{Response Relevancy} = \frac{0.9 + 0.8 + 0.85}{3} \approx 0.85$$

6.3.2.2 Faithfulness

Faithfulness measures the factual consistency of a response, aligned with the retrieved context. It evaluates whether all claims in the generated response can be supported by the retrieved context. Scores range from 0 to 1, with higher values indicating better alignment between the response and the context. This metric is particularly important for ensuring that the generated answers are factually accurate and grounded in the retrieved information.

The calculation involves the following steps:

1. **Identify Claims:** Extract all factual claims from the response using LLM.
2. **Verify Claims:** Check if each claim can be inferred or supported by the retrieved context by again using an LLM as a judge.
3. **Compute Faithfulness:** Calculate the ratio of supported claims to the total number of claims:

$$\text{Faithfulness} = \frac{\text{Number of Supported Claims}}{\text{Total Number of Claims}}$$

Below an example is provided along with its setup, showcasing the calculation of the metric:

- **User Input:** "What is the history of the White Tower in Thessaloniki?"

- **Retrieved Context:** "The White Tower of Thessaloniki is a historical monument built in the 15th century during the Ottoman Empire. It was originally used as a fortress and prison. Today, it serves as a museum and is one of the city's most iconic landmarks."
- **High Faithfulness Response:** "The White Tower was built in the 15th century during the Ottoman Empire. It was used as a fortress and prison and now functions as a museum."
- **Low Faithfulness Response:** "The White Tower was built in the 16th century by the Byzantines. It was used as a lighthouse and is now a museum."

The calculation steps are explained in detail below:

1. Identify Claims:

- **High Faithfulness Response:**
 - "The White Tower was built in the 15th century during the Ottoman Empire."
 - "It was used as a fortress and prison."
 - "It now functions as a museum."
- **Low Faithfulness Response:**
 - "The White Tower was built in the 16th century by the Byzantines."
 - "It was used as a lighthouse."
 - "It is now a museum."

2. Verify Claims:

- **High Faithfulness Response:**
 - Claim 1: Supported (matches context).
 - Claim 2: Supported (matches context).
 - Claim 3: Supported (matches context).
- **Low Faithfulness Response:**
 - Claim 1: Unsupported (incorrect century and builder).
 - Claim 2: Unsupported (no mention of the lighthouse in context).
 - Claim 3: Supported (matches context).

3. Compute Faithfulness:

- **High Faithfulness Response:**

$$\text{Faithfulness} = \frac{3}{3} = 1.0$$

- **Low Faithfulness Response:**

$$\text{Faithfulness} = \frac{1}{3} \approx 0.333$$

The interpretation here is that, the high faithfulness response scores perfectly because all its claims align with the retrieved context. The low faithfulness response scores poorly due to incorrect historical details and unsupported claims.

This metric ensures that responses are factually grounded in the retrieved context, making it essential for applications requiring high accuracy and reliability.

6.3.2.3 Semantic Similarity

Semantic Similarity measures the degree of semantic alignment between a generated response and a reference (ground truth) answer. This metric evaluates how closely the meaning of the response matches the reference, using embedding models to capture semantic relationships. Scores range from 0 to 1, with higher values indicating better alignment. Semantic Similarity is particularly useful for assessing the quality of responses in terms of their conceptual relevance and depth, without requiring exact textual matches. The calculation involves the following steps:

1. **Vectorize Text:** Convert both the reference and the generated response into dense vector representations using an embedding model.
2. **Compute Cosine Similarity:** Calculate the cosine similarity between the two vectors to measure their semantic alignment.

The formula for cosine similarity is:

$$\text{Semantic Similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Where:

- **A:** Vector representation of the reference.
- **B:** Vector representation of the generated response.
- **·:** Dot product of the vectors.
- **||A||** and **||B||:** Magnitudes of the vectors.

Below an example is provided along with its setup, showcasing the calculation of the metric:

- **Reference:** "Thessaloniki is a vibrant city in Greece, known for its rich history, the White Tower, and its lively waterfront. It is also a major cultural and economic hub in the Balkans."
- **High Similarity Response:** "Thessaloniki, a bustling city in Greece, is famous for its historical landmarks like the White Tower and its scenic waterfront. It plays a significant role in the cultural and economic life of the Balkans."
- **Low Similarity Response:** "Greece is a beautiful country with many islands and ancient ruins. Athens is its capital and a popular tourist destination."

The calculation steps are explained in detail below:

1. **Vectorize Text:** Use an embedding model (e.g., sentence-transformers) to convert the reference and responses into vectors. Example embeddings (hypothetical values):

Reference (**A**): $[0.8, 0.2, 0.5, \dots]$

High Similarity Response (**B**₁): $[0.75, 0.25, 0.45, \dots]$

Low Similarity Response (**B**₂): $[0.3, 0.7, 0.1, \dots]$

2. **Compute Cosine Similarity:**

- **High Similarity Response:**

$$\cos(\theta_1) = \frac{\mathbf{A} \cdot \mathbf{B}_1}{\|\mathbf{A}\| \|\mathbf{B}_1\|} = \frac{0.8 \times 0.75 + 0.2 \times 0.25 + 0.5 \times 0.45 + \dots}{\sqrt{0.8^2 + 0.2^2 + 0.5^2 + \dots} \times \sqrt{0.75^2 + 0.25^2 + 0.45^2 + \dots}}$$

$$\cos(\theta_1) \approx 0.92$$

- **Low Similarity Response:**

$$\cos(\theta_2) = \frac{\mathbf{A} \cdot \mathbf{B}_2}{\|\mathbf{A}\| \|\mathbf{B}_2\|} = \frac{0.8 \times 0.3 + 0.2 \times 0.7 + 0.5 \times 0.1 + \dots}{\sqrt{0.8^2 + 0.2^2 + 0.5^2 + \dots} \times \sqrt{0.3^2 + 0.7^2 + 0.1^2 + \dots}}$$

$$\cos(\theta_2) \approx 0.35$$

6.3.2.4 Factual Correctness

The Factual Correctness metric, compares the generated response and a reference (ground truth) and checks whether they are aligned in terms of factual accuracy. It breaks down the

response into claims and evaluates whether the claims in the response are supported by the reference, using precision, recall, or F1 score to quantify the overlap. In this thesis the default mode was selected for the Factual Correctness metric which is the F1. Scores range from 0 to 1, with higher values indicating better factual alignment. The metric is particularly useful for applications requiring high accuracy, such as historical question answering or fact-based dialogue systems.

The calculation involves the following steps:

1. **Claim Decomposition:** Break down the response and reference into atomic claims using LLM.
2. **Claim Support Verification:** Verify whether each claim in the response is supported by the reference using LLM.
3. **Compute Metrics:** Calculate precision, recall, or F1 score based on the verification results.

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Where:

- **True Positives (TP):** Claims in the response that are supported by the reference.
- **False Positives (FP):** Claims in the response that are not supported by the reference.
- **False Negatives (FN):** Claims in the reference that are missing in the response.

Below an example is provided along with its setup, showcasing the calculation of the metric:

1. **Claim Decomposition:**

• **Reference Claims:**

- "Thessaloniki is a major port city in Greece."
- "Thessaloniki is known for its Byzantine walls."

- "Thessaloniki is known for the White Tower."
- "Thessaloniki has a vibrant cultural scene."
- "Thessaloniki was founded in 315 BC by Cassander."

- **Response Claims:**

- "Thessaloniki is a port city in Greece."
- "Thessaloniki is famous for the White Tower."
- "Thessaloniki is famous for its Byzantine walls."
- "Thessaloniki was established in 315 BC."

2. Claim Support Verification:

- **True Positives (TP):**

- "Thessaloniki is a port city in Greece."
- "Thessaloniki is famous for the White Tower."
- "Thessaloniki is famous for its Byzantine walls."
- "Thessaloniki was established in 315 BC."

- **False Positives (FP):** None (all response claims are supported).

- **False Negatives (FN):**

- "Thessaloniki has a vibrant cultural scene."
- "Thessaloniki was founded by Cassander."

3. Compute Metrics:

- **Precision:**

$$\text{Precision} = \frac{4}{4 + 0} = 1.0$$

- **Recall:**

$$\text{Recall} = \frac{4}{4 + 2} \approx 0.666$$

- **F1 Score:**

$$\text{F1 Score} = 2 \times \frac{1.0 \times 0.666}{1.0 + 0.666} \approx 0.8$$

We can interpret the example above with the following:

- The response achieves perfect precision because all its claims are supported by the reference.
- The recall is lower because the response misses some details from the reference (e.g., cultural scene and Cassander).
- The F1 score balances precision and recall, providing a holistic measure of factual correctness.

6.4 Results

This section presents the results of every metric evaluation of the proposed RAG system framework. The experiments are initially structured as "Local Models Experiments" which use locally loaded models provided from Hugging Face (Llama3.2 3B Instruct LLM and BGE-M3 embedding model) and as "OpenAI Models Experimentation" with models provided from OpenAI (gpt4o-mini LLM and "text-embedding-3-small" model). Those experiments are specifically designed to measure the efficiency and effectiveness of the generation and retrieval part of each architecture. As previously mentioned, to evaluate the results, RAGAS library (provides metrics and evaluation implementations), llamaindex (provides RAG implementations and integrations) and OpenAI API (used for the LLM-as-a-judge in metrics that require it, along with embedding model) were extensively used. Moreover all those libraries are used in Python language and with the usage of a PostgreSQL database with PGVector plugin as mentioned in Section 6.2.2. Furthermore, due to the fact that GraphRAG is a completely separate library which does not use llamaindex for its integration, some implementation constraints exist. Those constraints forced the GraphRAG experiment to only be performed with OpenAI models and not with locally loaded ones. It must be noted that, all architectures were tested with top-k retrieved chunks parameter set to 5, as an increase would lead to big performance cost in terms of execution times. Finally all experiments use the same prompting template which is thoroughly explained in 5.1.3 Section.

6.4.1 Local Models Experimentation

As explained in Section 5.2.1 Baseline architecture was implemented to provide a benchmark for the other architectures as it uses a straightforward and efficient retrieval methodology through the use of BM25 retriever. Additionally it is considered the most cost efficient architecture as it involves only sparse vectors for retrieval, thus not needing any additional embedding model for its execution and subsequently reducing costs in terms of execution time and with reduced API requests to LLM providers. The Baseline's BM25 approach excels at ranking documents based on the exact terms in a query, however, it struggles when synonyms or more nuanced semantic matches are needed, which partly explains why it lags behind Hybrid or HyDE in certain recall metrics. As seen in Table 6.2 Baseline's performance when combined with the aforementioned local LLM model, is quite satisfactory, considering the fact that the computational cost is the lowest of all architectures with 4492 seconds or about 1 hour and 15 minutes. Even though in the Average Retrieval Quality metric, it scores the lowest from the other architectures, it manages to stick within 1% difference from the architecture with the highest Average Generative Quality metric. In more detail it scores within 1% from the best

scoring architectures in Semantic Similarity and Answer Relevancy and scores 3% less than the best performing architecture in Factual Correctness.

The Hybrid architecture combines part of the Baseline, namely the BM25 retriever and enhances it with the usage of dense vector representations with the help of embedding models as explained in Section 5.2.2. It improves retrieval quality to 0.594 on average, demonstrating higher Context Entity Recall than every other architecture with 0.523. It also balances out other retrieval metrics well, though it does not outperform HyDE or Automerge at the top end of certain metrics. Performing well on retrieval, is logical, considering that Hybrid architecture captures both exact matches and semantically similar passages, which is particularly useful for queries that do not exactly match the vocabulary or phrasing in the data. On the generation side, Hybrid performs similarly to the Baseline method at 0.654 Average Generative Quality with less than 1% deviation in Answer Relevancy. This is accompanied by competitive results in Factual Correctness (0.419) and Semantic Similarity (0.807). Its computational cost rises slightly to 5729 seconds or about 1 hour and 36 minutes, reflecting the added complexity of combining different retrieval strategies along with the query augmentation (1 original query and 3 augmented queries) that is performed during the fusion of the retrieved chunks. Overall it seems that Hybrid approach is slightly better than Baseline with the local models, but considering the computational added cost, Baseline seems preferable than the Hybrid one.

Automerge as explained in Section 5.2.3 is designed around iterative merging of retrieved nodes and hierarchical chunk structures. While this seems to boost precision, the iterative merges may not always capture every relevant variation of a user's query. As a result, its average retrieval rank can trail behind HyDE architecture in contexts where an initial chunk or parent node lacks coverage of the entire query. Average Retrieval Quality stands at about 1% from the Hybrid architecture experiment and more specifically at 0.603, which benefits greatly from its Context Precision without Reference that is about 0.945 and scores highest in this metric from the rest of the architectures. It also leads in Non LLM Context Recall at about 0.297, which is contradicting with LLM Context recall which is the lowest among all architectures at 0.515. As explained in Section 6.3.1.2 the difference between LLM and Non LLM Context recall is that the first one uses LLM which can catch broader semantic relationships and synonyms and the Non LLM version explicitly relies on text similarity. So the explanation here is probably that this architecture fails to include semantically relevant information which is not directly using the same text and this is further supported by the low Context Entity Recall which is even lower than Baseline (0.471). Automerge's Average Generative Quality is 0.610 which is the worst performing architecture in this aspect. Only, benefiting from the highest Semantic Similarity (0.809) and best Factual Correctness (0.421) among all four approaches doesn't seem to be enough as it lags greatly behind in Faithfulness (0.466). This may suggest that performing well on Recall metrics in the retrieval part is really important, to enable the model perform also well in the generation part. Automerge completes the evaluations in about 5091 seconds

or about 1 hour and 25 minutes, making it somewhat more expensive than Baseline but more efficient than Hybrid in runtime.

HyDE’s usage of LLM to generate a hypothetical text in response to the user’s query before the actual retrieval step, seemingly helps it achieve the highest Average Retrieval Quality of 0.625, which is 2 % above the AutoMerge architecture. This is notably boosted by its strong LLM Context Recall and Context Precision with Reference with values 0.743 and 0.803 accordingly, the best scoring among all architectures used with local models. HyDE also falls behind by a little margin in Context Entity Recall from the Hybrid architecture and scores the second best Non LLM Context Recall after AutoMerge, with a value of 0.212. In general this architecture gains satisfactory results across all recall metrics which is probably the reason of the high scores it also gains in the Generative Quality metrics. More specifically, HyDE achieves the best overall generative performance of 0.662, propelled by high Faithfulness (0.642) and generally robust scores on the other metrics, suggesting that its method of leveraging hypothetical documents supports more coherent and accurate generative outputs in accordance to the context provided. It does, however, come at a higher computational cost of approximately 8812 seconds or about 2 hours and 27 minutes. Its higher computational cost stems mostly from the additional queries needed for generating the hypothetical documents used for retrieval, but that overhead translates into more thorough retrieval coverage and on-target answers.

Metric	Baseline	Hybrid	Automerge	HyDE
Retrieval Quality				
Context Precision w/o Reference	0.824	0.873	0.945	0.846
Context Precision w Reference	0.653	0.764	0.788	0.803
LLM Context Recall	0.654	0.612	0.515	0.742
Non LLM Context Recall	0.208	0.198	0.297	0.212
Context Entity Recall	0.477	0.523	0.471	0.520
Average Retrieval Quality	0.563	0.594	0.603	0.625
Generative Quality				
Factual Correctness	0.396	0.419	0.421	0.370
Faithfulness	0.574	0.555	0.466	0.642
Semantic Similarity	0.806	0.807	0.809	0.801
Answer Relevancy	0.834	0.834	0.745	0.833
Average Generative Quality	0.653	0.654	0.610	0.662
Other				
Computational Cost (in seconds)	4492.241	5729.401	5090.602	8812.045

TABLE 6.2: Evaluation Results Across Different RAG Architectures with locally loaded models (Llama3.2 3B Instruct Quantized and BGE-M3)

Taking into account the results from the experiments with locally loaded models, HyDE emerges as the best performing architecture overall. It achieves the highest Average Retrieval Quality

at 0.625, with the highest Context Precision with Reference and LLM Context Recall, indicating an exceptional ability to identify and retrieve relevant documents. Although this approach comes at a higher computational cost, the trade-off is justified by the more coherent and accurate generative outputs it produces, suggesting that HyDE's strategy of leveraging hypothetical documents offers a substantial advantage in both precision and quality of the responses. This makes it a compelling choice for applications where accuracy and contextual relevance are essential, even if it requires more computational resources.

But, the fact that Average Generative Quality metrics in all architectures, except the Automerger, score within 1%, suggests that probably the locally loaded models and more specifically the LLM, reached their performance capabilities and this can be also supported by the high deviation of Retrieval metrics which essentially tend to improve more in comparison to the Baseline (Lowest 0.563 with Baseline and Highest 0.625 with HyDE). The next set of experiments targets this specific drawback by using LLM models through the OpenAI provider and exploring the capabilities of those architectures with stronger LLM models than the ones locally used.

6.4.2 OpenAI Models Experimentation

Up to this point experimentation was performed with locally loaded models such as LLama3.2 3B Instruct Quantized (LLM) and BGE-M3 (Embedding Model) and the results were presented in Section 6.4.1. But it became apparent that the architectures tested previously, probably due to the LLM used, were mostly not so different in terms of performance in the generation part even though substantial differences could be seen in the retrieval part of the architectures. This led to the experimentation with different more capable models and more specifically from OpenAI. In order to perform similar comparisons the most cost efficient LLM from OpenAI was used which was the "gpt4o-mini" along with the corresponding cost efficient embedding model available "text-embedding-3-small".

Baseline architecture even though not topping any of the retrieval metrics, manages to stay 5% behind the Hybrid architecture in Average Retrieval Quality, which scores the highest. Its precision metrics are quite lower than the Hybrid ones but achieves satisfactory results in all recall metrics most notably achieving 0.476 in Context Entity Recall, less than 1% below the highest performing architecture in this category, which is HyDE. An important thing to consider about the retrieval part of this architecture is that the retrieval part has not changed compared to the same experiment in Table 6.2. The small differences seen in the retrieval metric values (less than 1%) are a result of the non deterministic nature of the LLM-as-a-judge used in most metrics, which can provide slightly different output with the same input. The only metric that does not use LLM for its evaluation is Non LLM Context Recall, thus the value is the

same on both experiments. In terms of generative performance, Baseline performs strongly, closely competing with HyDE and outperforming Hybrid and AutoMerge in most of those metrics and achieves the second best Average Generative Quality among all architectures. In Factual Correctness, Baseline is higher than Hybrid, Automerge and GraphRAG with 0.424 but remains below HyDE which has 0.458. Again, in faithfulness, Baseline is better than Hybrid and Automerge but lower than HyDE and mainly GraphRAG which is the best performing architecture in this metric. In terms of Semantic Similarity, Baseline performs similarly to all architectures (with the exception of GraphRAG). Moreover in Answer Relevancy, Baseline has the highest value among all architectures. Finally, Baseline’s computational cost in terms of execution time is the lowest of all architectures, ensuring it remains a viable option when computational efficiency is paramount. This also comes in par with what the locally loaded model experiments indicated.

The Hybrid Vectors approach, performs satisfactory overall, achieving the highest context precision without a reference with 0.972, context precision with reference with 0.797, and a strong average retrieval quality with 0.610. This suggests that combining dense and sparse retrieval methods helps retrieve relevant information with high precision. In recall metrics, it falls behind the best performing architectures especially in LLM Context Recall which is 13% less than HyDE which scores the best with 0.744. In terms of generative quality, its average is 5% less than the best performing architecture HyDE and slightly lower than the baseline, with factual correctness at 0.387. This indicates that while retrieval precision is high, the fusion of multiple retrieval sources may introduce some noise or inconsistencies in generation. Its computational cost at 2737.739 seconds or about 45 minutes is slightly higher than the baseline but still considered efficient.

For AutoMerge, retrieval quality is generally underperforming, due to the low scores in metrics such as LLM Context Recall and Context Entity Recall, a pattern which was also observed in the experiments with the locally loaded models. The pattern seems consistent across different LLM models and again we see contradictory results between LLM Context Recall which performs the worst across all architectures and Non LLM Context Recall which performs the best. This further supports the claim made in Section 6.4.1 that this architecture fails to include semantically relevant information into its context. Its generative performance is slightly lower than Baseline and Hybrid, with an average quality of 0.635, possibly due to reasons mentioned above, as it is clear by now that recall plays a big role to the generative results. In combination with its computational cost of 3150.780 seconds or about 52 minutes, it does not seem as the go-to selection for this specific tourism domain scenario as the Baseline performs generally better in both retrieval and generation parts with less computational cost.

HyDE’s retrieval quality is overall strong, with a context precision of 0.839 without reference

and 0.665 with reference, while achieving the highest LLM context recall at 0.744 and the highest Context Entity Recall at 0.482 slightly ahead of the Hybrid architecture. This suggests that its generated hypothetical documents used in the embeddings are effective at recalling relevant documents and not missing relevant information, even if precision isn't as high as Hybrid Vectors. The generative quality is the highest overall at 0.697, with the best factual correctness score of 0.458, indicating that the approach enhances factual consistency in answers. This architecture also achieves the best Semantic Similarity metric with a value of 0.818. In comparison to the same architecture with the locally loaded models, it can be seen that in general HyDE architecture independently of the models used, achieves high recall in terms of retrieval and high generation metrics. However, the computational cost is relatively higher at 3725.630 seconds or about 1 hour and 2 minutes, but seems ideal for applications where high-generation accuracy is critical.

As mentioned in Section 5.2.5 the Local GraphRAG architecture uses as context, local community summaries which are generated from LLMs. This means that the retrieved parts are not chunks from the processed documents but generated summaries by LLM and more specifically the "gpt4o-mini" which depending to the communities, may be inaccurate and also use semantically different tokens. Thus the retrieval metrics highlighted in GraphRAG architecture in Table 6.3 are provided only as a reference and not for direct comparisons with the other architecture as they are not comparing the same things. This can be supported by Non LLM Context which relies on string similarities on retrieved parts and seen, it can hugely differentiate when the LLM of GraphRAG produces a summary thus not using same terms as the chunks would include. Its generative quality seems balanced, achieving the highest faithfulness score of 0.758, meaning responses adhere strongly to retrieved contexts, which in this case are the generated community summaries. Despite a lower factual correctness score of 0.369, the strong faithfulness suggests it generates well-grounded answers. Moreover it scores the third best Average Generative Quality score after HyDE and Baseline. Its computational cost is by far the highest of the OpenAI Models experiments at 5713.413 seconds or about 1 hour and 35 minutes making other architectures more appealing for selection.

Metric	Baseline	Hybrid	Automerge	HyDE	GraphRAG
Retrieval Quality					
Context Precision w/o Reference	0.825	0.972	0.950	0.839	0.726
Context Precision w Reference	0.666	0.797	0.667	0.665	0.529
LLM Context Recall	0.656	0.611	0.441	0.744	0.599
Non LLM Context Recall	0.208	0.189	0.255	0.212	0.047
Context Entity Recall	0.476	0.481	0.421	0.482	0.313
Average Retrieval Quality	0.566	0.610	0.547	0.588	0.443
Generative Quality					
Factual Correctness	0.424	0.387	0.373	0.458	0.369
Faithfulness	0.638	0.540	0.519	0.672	0.758
Semantic Similarity	0.818	0.811	0.808	0.818	0.751
Answer Relevancy	0.852	0.844	0.840	0.841	0.824
Average Generative Quality	0.683	0.646	0.635	0.697	0.676
Other					
Computational Cost (in seconds)	2663.285	2737.739	3150.780	3725.630	5713.413

TABLE 6.3: Evaluation Results Across Different RAG Architectures with OpenAI models (GPT4o-mini and text-embedding-3-small). Retrieval Quality metrics in GraphRAG not applicable.

In general HyDE, seems again the overall best performer with strong performance across all retrieval metrics and with clear advantage in the recall ones while also performing the best in most of the generative metrics. Additionally, the computational cost seems small when considering the uplift in generative metrics. In the next section, the best performing architecture of both locally loaded and OpenAI models will be compared, which is actually HyDE both in terms of performance and computational cost.

6.4.3 Best Architecture per Model Comparison

The comparison between HyDE Local and HyDE OpenAI, the best approaches from the 2 different experimentations presented previously, reveals a trade-off between retrieval effectiveness and generative performance, influenced by the quality of the embedding models and the strength of the LLM used in each setup. HyDE Local achieves a higher average retrieval quality of 0.625 compared to 0.588 for HyDE OpenAI. This is further reinforced by its higher LLM Context Precision with Reference at 0.803 versus 0.665, as well as its higher Context Entity Recall at 0.520 compared to 0.482. These results suggest that the locally loaded embedding model (BGE-M3) is more effective at capturing semantic relevance likely due to better domain adaptation or due to being a more capable model in general. The improved retrieval precision with reference also indicates that the retrieved documents are more aligned with ground truth answers in HyDE Local, potentially reducing the noise in the input provided to the LLM for

generation. Despite its weaker retrieval performance, HyDE OpenAI significantly outperforms its local counterpart in generative quality. It achieves a much higher factual correctness score of 0.458 compared to 0.370, demonstrating that the OpenAI LLM is more capable of producing factually accurate responses based on the retrieved context. Additionally, it leads in faithfulness at 0.672 versus 0.642, meaning that its generated answers are more consistent with the retrieved information and less prone to hallucination. Semantic similarity and answer relevancy are also slightly better in the OpenAI setup, with values of 0.818 and 0.841, respectively, indicating that the responses are more aligned with user queries while maintaining coherence. These improvements suggest that OpenAI’s stronger language model compensates for the slightly lower retrieval performance by better interpreting the retrieved documents and generating more accurate and relevant responses.

The total execution time is not directly comparable due to differences in computational environments. The OpenAI-based approach benefits from offloading processing to cloud infrastructure, while the local model operates on a more limited system with an RTX 3060 GPU and a Ryzen 3600 processor. The lower execution time for HyDE OpenAI is a result of API-based execution rather than an inherent efficiency gain in the model itself. Additionally, OpenAI’s approach incurs a monetary cost per request, whereas the local model avoids API fees but is limited by hardware constraints.

Metric	HyDE Local	HyDE OpenAI
Retrieval Quality		
Context Precision w/o Reference	0.846	0.839
Context Precision w Reference	0.803	0.665
LLM Context Recall	0.742	0.744
Non LLM Context Recall	0.212	0.212
Context Entity Recall	0.520	0.482
Average Retrieval Quality	0.625	0.588
Generative Quality		
Factual Correctness	0.370	0.458
Faithfulness	0.642	0.672
Semantic Similarity	0.801	0.818
Answer Relevancy	0.833	0.841
Average Generative Quality	0.662	0.697
Other		
Total Time (seconds)	8812.045	3725.630

TABLE 6.4: Comparison of HyDE Local and HyDE OpenAI across Retrieval and Generative Metrics.

Overall, HyDE Local excels in retrieval due to a better embedding model, ensuring higher precision and recall (with the exception of LLM Context Recall) in fetching relevant documents,

but its generative performance is weaker due to the constraints of the local LLM. HyDE OpenAI, on the other hand, compensates for slightly weaker retrieval with a significantly stronger LLM, leading to better generative performance, particularly in factual correctness and faithfulness. The choice between the two depends on the use case: if high retrieval precision is critical, HyDE Local may be preferable, but for applications requiring high-quality generation, HyDE OpenAI is the better option. The experimental results presented in Chapter 6 directly contribute to the benchmarking of State-of-the-Art RAG architectures discussed in Chapter 1 (see Section 1.3)

To sum up, this chapter describes the evaluation process of the RAG system for tourism enhancement presented in this thesis, as well as the results of the evaluation for all different architectures. The upcoming chapter discusses the aforementioned results to reach useful conclusions.

Chapter 7

Conclusions & Future Work

In the previous chapter, the experimental results of the Retrieval-Augmented Generation (RAG) system for tourism enhancement were presented and analyzed. This chapter discusses these findings in depth, evaluating the effectiveness of the proposed architectures and identifying areas for future research and improvements.

7.1 Retrieval and Generative Performance Trade-offs

This thesis explored five different RAG architectures: Baseline, Hybrid Vectors, AutoMerge, HyDE, and GraphRAG to determine their effectiveness in improving retrieval and generation processes within the tourism domain. The experimentation was conducted using both locally loaded models and OpenAI's models, providing insights into their comparative performance under different computational setups. Moreover, the results indicate that recall seems to play a more significant role than precision in improving generative quality, particularly in ensuring the LLM has access to all necessary context for accurate response synthesis. Architectures that performed well in LLM Context Recall, such as HyDE, demonstrated superior generative performance, while those with lower recall, like AutoMerge, struggled in producing coherent and factually grounded responses.

The Baseline architecture, which relies on BM25 sparse retrieval, provided a strong and computationally efficient benchmark. It demonstrated competitive generative performance despite lacking semantic retrieval capabilities, proving that traditional sparse retrieval methods remain viable in well-structured domains where queries closely match document content.

The HyDE architecture though, consistently achieved the highest generative performance, regardless of whether locally loaded models or OpenAI models were used. Its ability to generate

hypothetical document embeddings before retrieval significantly improved recall metrics, ensuring that relevant information was retrieved even when it was not explicitly present in the original query. This resulted in the highest scores in factual correctness, faithfulness, and semantic similarity, demonstrating that improving recall directly enhances generative accuracy.

Conversely, Hybrid Vectors exhibited the highest context precision but fell behind in recall, leading to lower generative quality compared to HyDE. While its fusion of sparse and dense retrieval improved the likelihood of retrieving relevant documents, the lower recall limited its ability to provide complete context to the LLM, ultimately affecting factual correctness and faithfulness.

AutoMerge, despite its hierarchical retrieval strategy, struggled with recall, showing the lowest LLM Context Recall score across all architectures. This suggests that its merging process might exclude relevant but less explicitly connected documents, leading to fragmented contextual input for generation. Consequently, it achieved the lowest generative performance, reinforcing the idea that recall-driven architectures perform better in RAG settings where response coherence and factual grounding are critical.

GraphRAG introduced a unique knowledge graph-based retrieval mechanism, which produced highly faithful responses. However, its recall metrics were difficult to interpret due to the nature of its retrieval process, which generates community-based summaries rather than retrieving exact document chunks. This difference in retrieval methodology made direct comparisons challenging, but the model excelled in faithfulness, highlighting the potential of graph-based retrieval in structuring knowledge more effectively for response synthesis.

7.2 Local vs. OpenAI Model Comparisons

The comparison between HyDE Local and HyDE OpenAI revealed an important balance between retrieval effectiveness and generative strength. The locally loaded BGE-M3 embedding model outperformed OpenAI's embedding model in terms of retrieval recall and precision, suggesting it was better suited for the domain-specific data used in the experiments. However, the OpenAI LLM significantly outperformed the locally loaded Llama model in generation, achieving higher factual correctness and faithfulness scores. This, emphasizes the importance of combining strong recall-driven retrieval with a powerful LLM for optimal performance. While a high-quality embedding model ensures that all relevant documents are retrieved, an advanced LLM is necessary to generate accurate and contextually appropriate responses.

In general, Baseline retrieval with BM25 remained an efficient and competitive approach, particularly in scenarios where computational cost is a concern. Despite lacking the deep semantic retrieval of hybrid or dense-based approaches, its ability to rank documents effectively

allowed it to remain a viable choice when precision-based retrieval is sufficient. While HyDE achieved the best generative results, it also incurred the highest computational costs due to the additional retrieval and augmentation steps involved. AutoMerge demonstrated that hierarchical retrieval can be computationally efficient but may suffer from lower recall. Similarly, GraphRAG required the longest execution times, primarily due to the complexity of generating and retrieving structured knowledge summaries. The findings suggest that retrieval complexity must be balanced with system constraints, particularly in real-time applications. For tourism applications where user experience depends on response latency, a trade-off must be made between retrieval effectiveness and computational overhead.

7.3 Future Work

The results of this thesis open several avenues for further exploration. Future research should focus on optimizing recall-driven retrieval methods, exploring adaptive retrieval mechanisms, and enhancing system efficiency without compromising response quality.

The results demonstrated that high recall is more beneficial than high precision in improving generative performance. Future work should investigate adaptive retrieval mechanisms that dynamically adjust retrieval strategies based on query complexity. For simple queries, sparse retrieval methods such as BM25 may be sufficient, whereas more complex queries could trigger multi-hop dense retrieval or query expansion techniques to maximize recall.

The higher retrieval performance of BGE-M3 over OpenAI's embedding model suggests that probably, domain-adapted embeddings play a crucial role in improving retrieval quality. Fine-tuning existing embedding models on tourism-specific data could further enhance recall and retrieval alignment, leading to better generative outputs. Additionally, integrating hybrid retrieval with personalized embeddings could refine retrieval even further, allowing the system to adapt to user-specific preferences and historical interactions.

While OpenAI's GPT-4o-mini significantly outperformed locally loaded Llama models in generation, its reliance on API calls introduces cost constraints. Future work should explore fine-tuning smaller, locally deployable models that can match OpenAI's performance while reducing dependency on external APIs. Distillation techniques could be applied to create lightweight versions of high-performance LLMs, offering a middle ground between retrieval accuracy and generative strength without incurring high operational costs.

The current study focused on monolingual retrieval and generation. Given the global nature of tourism, future research should investigate multi-language retrieval models that support

cross-lingual document retrieval and generation. Expanding the dataset to include multilingual sources would allow better evaluation of RAG systems in diverse linguistic contexts. Additionally, while this research was conducted in the tourism domain, the architectures explored in this thesis could be extended to other information-dense fields such as healthcare, legal document retrieval, or academic research. Exploring the adaptability of RAG architectures across multiple domains would provide broader insights into their scalability and effectiveness.

Finally, while this study was conducted in a controlled experimental environment, deploying the system in real world tourism applications would provide valuable information on usability, user satisfaction, and practical performance. Future work should incorporate user feedback loops, interactive evaluation methods, and continuous model updates to refine retrieval strategies based on real-world usage patterns.

7.4 Final Remarks

This thesis has provided an in-depth exploration of retrieval-augmented generation architectures, analyzing their performance in a tourism-specific RAG system. The findings confirm that recall plays a critical role in retrieval quality and significantly impacts generative performance, particularly in domains where factual consistency is essential. The research highlights the importance of balancing retrieval accuracy, LLM strength, and computational efficiency to optimize system performance. By refining recall-driven retrieval methods, fine-tuning domain-specific embedding models, and exploring cost-effective alternatives for generative models, future work can further enhance the effectiveness and scalability of RAG systems across multiple domains.

Bibliography

- [1] Daron Acemoglu and Pascual Restrepo. 2018. Artificial intelligence, automation, and work. In *The economics of artificial intelligence: An agenda*. University of Chicago Press, 197–236.
- [2] Negar Arabzadeh, Xinyi Yan, and Charles LA Clarke. 2021. Predicting efficiency/effectiveness trade-offs for dense vs. sparse retrieval strategy selection. In *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*. 2862–2866.
- [3] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *arXiv preprint arXiv:2310.11511* (2023).
- [4] Mario Casillo, Fabio Clarizia, Giuseppe D’Aniello, Massimo De Santo, Marco Lombardi, and Domenico Santaniello. 2020. CHAT-Bot: A cultural heritage aware teller-bot for supporting touristic experiences. *Pattern Recognition Letters* 131 (2020), 234–243.
- [5] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. *arXiv preprint arXiv:2402.03216* (2024).
- [6] Jeffrey Cheng, Marc Marone, Orion Weller, Dawn Lawrie, Daniel Khashabi, and Benjamin Van Durme. 2024. Dated Data: Tracing Knowledge Cutoffs in Large Language Models. *arXiv preprint arXiv:2403.12958* (2024).
- [7] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. 2024. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024).
- [8] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. Ragas: Automated evaluation of retrieval augmented generation. *arXiv preprint arXiv:2309.15217* (2023).
- [9] Wenqi Fan, Yajuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on rag meeting llms: Towards retrieval-augmented

- large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 6491–6501.
- [10] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. 2024. Detecting hallucinations in large language models using semantic entropy. *Nature* 630, 8017 (2024), 625–630.
- [11] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2022. Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496* (2022).
- [12] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, et al. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *arXiv preprint arXiv:2311.05232* (2023).
- [13] Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C Park. 2024. Adaptive-rag: Learning to adapt retrieval-augmented large language models through question complexity. *arXiv preprint arXiv:2403.14403* (2024).
- [14] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmllingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736* (2023).
- [15] Ziyang Jiang, Xueguang Ma, and Wenhui Chen. 2024. Longrag: Enhancing retrieval-augmented generation with long-context llms. *arXiv preprint arXiv:2406.15319* (2024).
- [16] Zhengbao Jiang, Frank F Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. *arXiv preprint arXiv:2305.06983* (2023).
- [17] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. *arXiv preprint arXiv:2004.04906* (2020).
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.
- [19] Jiwei Li, Michel Galley, Chris Brockett, Georgios P Spithourakis, Jianfeng Gao, and Bill Dolan. 2016. A persona-based neural conversation model. *arXiv preprint arXiv:1603.06155* (2016).

- [20] Bingquan Liu, Zhen Xu, Chengjie Sun, Baoxun Wang, Xiaolong Wang, Derek F Wong, and Min Zhang. 2017. Content-oriented user modeling for personalized response ranking in chatbots. *IEEE/ACM Transactions on Audio, Speech, and Language Processing* 26, 1 (2017), 122–133.
- [21] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting for retrieval-augmented large language models. *arXiv preprint arXiv:2305.14283* (2023).
- [22] Octavian-Mihai Machidon, Aleš Tavčar, Matjaž Gams, and Mihai Duguleană. 2020. CulturalERICA: A conversational agent improving the exploration of European cultural heritage. *Journal of Cultural Heritage* 41 (2020), 152–165.
- [23] Ben Mann, N Ryder, M Subbiah, J Kaplan, P Dhariwal, A Neelakantan, P Shyam, G Sastry, A Askell, S Agarwal, et al. 2020. Language models are few-shot learners. *arXiv preprint arXiv:2005.14165* 1 (2020).
- [24] Jinhu Qi, Shuai Yan, Wentao Zhang, Yibo Zhang, Zirui Liu, and Ke Wang. 2024. Research on Tibetan Tourism Viewpoints information generation system based on LLM. In *2024 12th International Conference on Intelligent Computing and Wireless Optical Communications (ICWOC)*. IEEE, 35–41.
- [25] Jinhu Qi, Shuai Yan, Yibo Zhang, Wentao Zhang, Rong Jin, Yuwei Hu, and Ke Wang. 2024. RAG-Optimized Tibetan Tourism LLMs: Enhancing Accuracy and Personalization. *arXiv preprint arXiv:2408.12003* (2024).
- [26] Stephen E Robertson, Steve Walker, Susan Jones, Micheline M Hancock-Beaulieu, Mike Gatford, et al. 1995. Okapi at TREC-3. *Nist Special Publication Sp* 109 (1995), 109.
- [27] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D Manning. 2024. Raptor: Recursive abstractive processing for tree-organized retrieval. *arXiv preprint arXiv:2401.18059* (2024).
- [28] Ronit Singhal, Pransh Patwa, Parth Patwa, Aman Chadha, and Amitava Das. 2024. Evidence-backed fact checking using RAG and few-shot in-context learning with LLMs. *arXiv preprint arXiv:2408.12060* (2024).
- [29] Karen Sparck Jones. 1972. A statistical interpretation of term specificity and its application in retrieval. *Journal of documentation* 28, 1 (1972), 11–21.
- [30] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query expansion with large language models. *arXiv preprint arXiv:2303.07678* (2023).
- [31] Zhan Wang, Lin-Ping Yuan, Liangwei Wang, Bingchuan Jiang, and Wei Zeng. 2024. Virtuwander: Enhancing multi-modal interaction for virtual tour guidance through large

- language models. In *Proceedings of the CHI conference on human factors in computing systems*. 1–20.
- [32] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems* 35 (2022), 24824–24837.
- [33] Qikai Wei, Mingzhi Yang, Jinqiang Wang, Wenwei Mao, Jiabo Xu, and Huansheng Ning. 2024. Tourllm: Enhancing llms with tourism knowledge. *arXiv preprint arXiv:2407.12791* (2024).
- [34] Thomas Wolf. 2020. Transformers: State-of-the-Art Natural Language Processing. *arXiv preprint arXiv:1910.03771* (2020).
- [35] Xiao Yang, Kai Sun, Hao Xin, Yushi Sun, Nikita Bhalla, Xiangsen Chen, Sajal Choudhary, Rongze Daniel Gui, Ziran Will Jiang, Ziyu Jiang, et al. 2024. CRAG–Comprehensive RAG Benchmark. *arXiv preprint arXiv:2406.04744* (2024).
- [36] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. In *CCF Conference on Big Data*. Springer, 102–120.
- [37] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, and Bin Cui. 2024. Retrieval-augmented generation for ai-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024).